

نکته: در ابتدا گزارش دو بخش اول به صورت لتکس نوشته شده بود ولی به دلیل قطعی اینترنت مجبور شدم اون رو به ورد بیاورم پس اگر جایی هنوز به صورت لتکس بود عذرخواهی میکنم .

## Part1)

بارگذاری سیگنال صوتی و گزارش فرکانس نمونه برداری

در گام نخست، فایل صوتی `teletext.wav` که شامل صدای فشرده شدن دکمه‌های تلفن می‌باشد، با استفاده از محیط MATLAB بارگذاری شد. این فایل در یک آرایه ذخیره شده و فرکانس نمونه برداری آن استخراج گردید. فرکانس نمونه برداری، یکی از مهم‌ترین پارامترهای تحلیل سیگنال بوده و نقش اساسی در تفسیر نتایج حوزه‌ی زمان و فرکانس ایفا می‌کند.

کد مورد استفاده برای بارگذاری سیگنال و استخراج پارامترهای اولیه به صورت زیر است:

```
[x, Fs] = audioread("Data/teletext.wav")
```

```
;N = length(x)
```

```
;Ts = 1/Fs
```

```
;t = (0:N-1)/Fs
```

در این کد، متغیر `x` شامل نمونه‌های سیگنال صوتی، `Fs` بیانگر فرکانس نمونه برداری، `N` تعداد کل نمونه‌ها و `t` محور زمان متناظر با سیگنال بر حسب ثانیه می‌باشد. تعریف محور زمان، امکان تحلیل دقیق سیگنال در حوزه‌ی زمان و تطبیق آن با تحلیل‌های حوزه‌ی فرکانس و زمان-فرکانس را فراهم می‌سازد.

فرکانس نمونه برداری فایل صوتی برابر با

```
Fs = 8000Hz
```

می‌باشد که نشان می‌دهد سیگنال با نرخ مناسبی برای تحلیل گفتار و سیگنال‌های تلفنی نمونه برداری شده است.

```
>> Q1_1
Fs = 8000 Hz
..
```

فرکانس نمونه‌برداری به‌دست‌آمده با استانداردهای متداول سیستم‌های مخابراتی تلفنی سازگار بوده و امکان تحلیل مؤلفه‌های فرکانسی سیگنال در بازه‌ی مورد نظر را فراهم می‌کند. در ادامه، با استفاده از تبدیل فوریه، تحلیل فرکانسی سیگنال انجام خواهد شد.

محاسبه تبدیل فوریه و تخمین طیف سیگنال پیوسته

در این مرحله هدف، محاسبه‌ی طیف فرکانسی سیگنال و رسم طیف توان با محور افقی بر حسب هرتز است.

از آن‌جا که سیگنال در اختیار ما به‌صورت گسسته‌زمان نمونه‌برداری شده است، خروجی محاسبه‌ی FFT مستقیماً نمایان‌گر  $X_{dis}(e^{j\omega})$  بوده و برای تخمین طیف متناظر پیوسته‌زمان باید مقیاس‌گذاری مناسب انجام شود.

طبق رابطه‌ی ارائه‌شده در صورت پروژه، تخمین تبدیل فوریه پیوسته از روی تبدیل فوریه گسسته به صورت زیر انجام می‌شود:

$$\omega = \Omega T_s \text{ با } X_{con}(j\Omega) = T_s * X_{dis}(e^{j\omega})$$

که در آن  $T_s = 1/F_s$  دوره‌ی نمونه‌برداری است. بنابراین پس از محاسبه‌ی FFT، مقدار طیف با ضریب  $T_s$  ضرب می‌شود

تا تخمین صحیح‌تری از طیف پیوسته به دست آید. این مقیاس‌گذاری اهمیت زیادی دارد؛ زیرا بدون آن، دامنه‌ی طیف به‌صورت وابسته به نرخ نمونه‌برداری ظاهر شده و امکان مقایسه‌ی درست انرژی/توان سیگنال در تحلیل پیوسته‌زمان کاهش می‌یابد.

دلیل استفاده از ضریب  $T_s$ : خروجی FFT تقریب انتگرال فوریه را در قالب یک جمع گسسته ارائه می‌دهد؛

در تبدیل پیوسته‌زمان، عنصر انتگرال  $dt$  وجود دارد و در تبدیل گسسته، معادل آن  $T_s$  است. به همین دلیل، برای تبدیل مقیاس DFT به تخمین CTFT باید ضریب  $T_s$  لحاظ شود.

بهبود نمایش طیف: حذف مؤلفه DC و صفرپدینگ

پیش از محاسبه‌ی FFT، مؤلفه‌ی DC (میانگین سیگنال) از سیگنال کم شد. در سیگنال‌های واقعی، وجود یک مؤلفه‌ی میانگین می‌تواند یک قله‌ی بزرگ در حوالی فرکانس صفر ایجاد کند و باعث شود مشاهده‌ی مؤلفه‌های مهم‌تر (مانند فرکانس‌های مربوط به دکه‌ها) سخت‌تر گردد.

بنابراین حذف میانگین، صرفاً برای شفاف‌تر شدن نمایش طیف و جلوگیری از غالب شدن قله‌ی نزدیک صفر انجام شده است و ماهیت اصلی فرکانس‌های موجود در سیگنال را تغییر نمی‌دهد.

همچنین برای محاسبه‌ی FFT، طول تبدیل به نزدیک‌ترین توان ۲ بزرگ‌تر از طول سیگنال انتخاب شد (صفرپدینگ).

این کار دو مزیت اصلی دارد:

\* کاهش زمان محاسبات: الگوریتم‌های FFT برای طول‌های توان ۲ بسیار سریع‌تر عمل می‌کنند.

\* ریزتر شدن شبکه فرکانسی: صفرپدینگ باعث افزایش تعداد نقاط طیف و در نتیجه افزایش تفکیک پذیری شبکه‌ی فرکانس (frequency grid) می‌شود؛

بنابراین مکان قله‌های طیفی خوانا تر و دقیق تر دیده می‌شود.

نکته مهم: صفرپدینگ به معنی اضافه کردن اطلاعات جدید به سیگنال نیست و رزولوشن واقعی فیزیکی را افزایش نمی‌دهد،

اما باعث می‌شود طیف به صورت نرم تر و با نمونه برداری فرکانسی متراکم تر نمایش داده شود؛ بنابراین برای خواندن دقیق فرکانس ها بسیار مفید است.

محور فرکانس بر حسب هرتز و استفاده از `fftshift`

خروجی FFT ذاتاً بر حسب اندیس های DFT و فرکانس زاویه ای گسسته  $\omega$  مرتب می‌شود؛ در حالی که در این پروژه لازم است محور افقی بر حسب هرتز ارائه گردد.

به همین دلیل، محور فرکانس با استفاده از  $F_s$  و طول تبدیل  $N\_FFT$  ساخته شد تا هر نمونه طیف به یک فرکانس

در واحد هرتز متناظر شود. همچنین از `fftshift` استفاده شد تا طیف دوطرفه به صورت متقارن حول صفر نمایش داده شود؛

به بیان دیگر، فرکانس صفر به مرکز منتقل می‌شود و بازه‌ی فرکانسی به شکل  $-F_s/2$  تا  $F_s/2$  نمایش داده خواهد شد. این نمایش دوطرفه برای تحلیل سیگنال های واقعی مناسب تر است زیرا در سیگنال های حقیقی،

طیف به صورت مزدوج-متقارن حول صفر ظاهر می‌شود و مشاهده‌ی این تقارن می‌تواند صحت تحلیل را نیز تأیید کند.

محاسبه طیف توان

پس از تخمین  $X\_con(j\Omega)$ ، برای بررسی شدت مؤلفه های فرکانسی، طیف توان محاسبه شد:

$$P(f) = |X\_con(j\Omega)|^2$$

استفاده از توان (به جای دامنه) دو دلیل اصلی دارد:

\* قله ها (فرکانس های غالب) واضح تر و قابل مقایسه تر می‌شوند.

\* در تحلیل سیگنال، توان معیار مناسب تری برای بیان سهم هر مؤلفه‌ی فرکانسی از انرژی/شدت سیگنال است.

در شکل 1 طیف توان تخمینی سیگنال `teletext.wav` با استفاده از FFT و مقیاس گذاری  $T_s$  (محور فرکانس بر حسب هرتز) رسم شده است. قله های اصلی طیف متناظر با فرکانس های موجود در صدای دکمه ها هستند

که در مراحل بعدی با استفاده از تحلیل زمان-فرکانس (اسپکتروگرام) مشخص خواهد شد هر قله در چه بازه‌ی زمانی فعال شده است.

```

N = length(x);
Ts = 1/Fs;

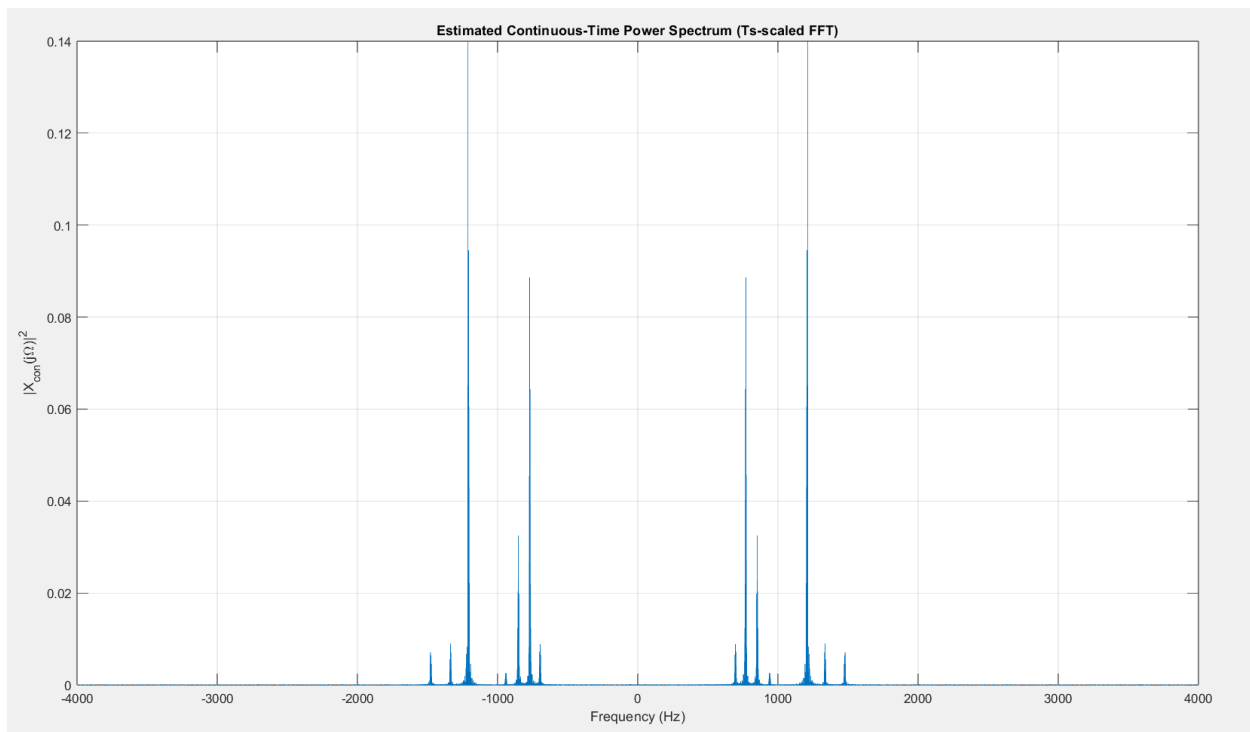
x0 = x - mean(x);
Nfft = 2^nextpow2(N);
Xdis = fft(x0, Nfft);
f = (-Nfft/2 : Nfft/2-1) * (Fs/Nfft);
Xdis_shift = fftshift(Xdis);
Xcon = Ts * Xdis_shift;

P = abs(Xcon).^2;

figure;
plot(f, P);
grid on;
xlabel("Frequency (Hz)");
ylabel("|X_{con}(j\Omega)|^2");
title("Estimated Continuous-Time Power Spectrum (Ts-scaled FFT)");

```

خروجی :



تحلیل زمان-فرکانس با Spectrogram و تنظیم پارامترها برای خواندن فرکانس‌ها

هدف این بخش، بررسی محتوای فرکانسی سیگنال در طول زمان است تا بتوان در هر لحظه تشخیص داد کدام فرکانس‌ها فعال هستند. از آن‌جا که صدای هر دکمه‌ی تلفن شامل دو مؤلفه‌ی فرکانسی همزمان است، نمایش زمان-فرکانس باید به اندازه‌ی کافی شفاف باشد تا این دو فرکانس به شکل دو قله‌ی مجزا قابل مشاهده و استخراج شوند. برای این منظور از تابع spectrogram در MATLAB استفاده شده که در واقع تبدیل فوریه‌ی زمان کوتاه (STFT) را محاسبه می‌کند.

پیش‌پردازش سیگنال

تبدیل سیگنال به مونو

در برخی فایل‌های صوتی، سیگنال به صورت استریو ذخیره می‌شود.

در این پروژه هدف استخراج فرکانس‌های دکمه‌هاست و اختلاف دو کانال معمولاً اهمیتی ندارد.

همچنین پردازش دوکاناله می‌تواند نمایش را شلوغ‌تر کند یا باعث شود دو طیف روی هم بیفتند.

بنابراین ابتدا سیگنال به مونو تبدیل می‌شود تا تحلیل ساده و پایدار باشد:

```
if size(x,2) > 1
```

```
;x = mean(x,2)
```

```
end
```

تابع آماده‌ی mean میانگین هر سطر را گرفته و دو کانال را به یک سیگنال واحد تبدیل می‌کند.

حذف مؤلفه DC (میانگین)

وجود مؤلفه‌ی DC باعث ایجاد یک قله‌ی بزرگ نزدیک 0 هرترز می‌شود که می‌تواند مقیاس رنگی اسپکتروگرام را خراب کند و فرکانس‌های مهم را کم‌رنگ نشان دهد.

به همین دلیل، میانگین سیگنال از آن کم می‌شود:

```
;x = x - mean(x)
```

این کار محتوای فرکانسی اصلی دکمه‌ها را تغییر نمی‌دهد، فقط نمایش را خواناتر می‌کند.

تنظیم پارامترهای spectrogram با هدف رزولوشن فرکانسی بالا

تابع spectrogram به‌طور خلاصه سیگنال را به قطعات کوتاه تقسیم می‌کند،

روی هر قطعه یک تبدیل فوریه می‌گیرد و سپس نتایج را کنار هم می‌چیند تا تغییرات فرکانس در زمان دیده شود.

پارامترهای کلیدی که باید تنظیم شوند عبارتند از:

طول پنجره (winLen)

طول پنجره مشخص می‌کند هر تبدیل فوریه روی چه تعداد نمونه انجام شود.

هرچه پنجره بزرگ‌تر باشد، رزولوشن فرکانسی بهتر می‌شود اما رزولوشن زمانی بدتر.

برای خواندن دقیق فرکانس‌های دوتایی دکمه‌ها، رزولوشن فرکانسی اهمیت بیشتری دارد،

پس پنجره نسبتاً بزرگ انتخاب شده است:

$$winLen = \text{round}(0.080 * Fs)$$

یعنی حدود 80 میلی‌ثانیه از سیگنال در هر فریم تحلیل می‌شود.

گام زمانی (hopLen) و هم‌پوشانی (noverlap)

hopLen فاصله‌ی شروع دو پنجره‌ی متوالی است. اگر این مقدار کوچک باشد، پنجره‌ها هم‌پوشانی زیادی دارند و تغییرات زمانی نرم‌تر دیده می‌شود.

هم‌پوشانی نیز برابر است با تفاوت طول پنجره و گام:

$$hopLen = \text{round}(0.010 * Fs)$$

$$noverlap = winLen - hopLen$$

در اینجا گام 10 میلی‌ثانیه است، بنابراین هم‌پوشانی زیاد شده و اسپکتروگرام در محور زمان پرش کمتری خواهد داشت (مخصوصاً برای تشخیص آغاز و پایان فشردن هر دکمه).

تعداد نقاط FFT (nfft)

nfft تعیین می‌کند که FFT با چند نقطه محاسبه شود. اگر nfft بزرگ‌تر از طول پنجره باشد، عملاً در حوزه فرکانس نمونه‌برداری متراکم‌تر خواهیم داشت (نمایش فرکانس نرم‌تر و قابل خواندن‌تر می‌شود).

این کار اطلاعات جدید تولید نمی‌کند، اما قله‌ها را دقیق‌تر روی محور فرکانس نمایش می‌دهد:

$$nfft = 2^{\text{nextpow2}(4 * winLen)}$$

تابع آماده‌ی nextpow2 کوچک‌ترین توان ۲ بزرگ‌تر از عدد داده‌شده را برمی‌گرداند تا محاسبات FFT سریع‌تر و استانداردتر انجام شود.

نوع پنجره (window)

برای کاهش spectral leakage (پخش شدن انرژی یک فرکانس روی همسایه‌ها)،

به جای پنجره مستطیلی از پنجره‌ی Hamming استفاده شده است:

`w = hamming(winLen, "periodic")`

تابع آماده‌ی hamming یک پنجره هموار تولید می‌کند که باعث کاهش لوب‌های جانبی در طیف و در نتیجه واضح‌تر شدن قله‌های فرکانسی می‌شود. گزینه‌ی "periodic" برای سازگاری با FFT

(حالت تحلیل طیفی) استفاده شده است.

محاسبه اسپکتروگرام و نمایش آن

محاسبه STFT با spectrogram

اکنون با پارامترهای تنظیم‌شده، spectrogram محاسبه می‌شود:

`[S, F, T] = spectrogram(x, w, noverlap, nfft, Fs, "yaxis")`

این دستور خروجی‌های زیر را می‌دهد:

\* S: ماتریس مختلط STFT (هر ستون یک بازه زمانی و هر سطر یک فرکانس است).

\* F: بردار فرکانس‌ها بر حسب هرتز.

\* T: بردار زمان‌ها بر حسب ثانیه.

گزینه‌ی "yaxis" باعث می‌شود خروجی فرکانس‌ها به صورت مناسب برای نمایش روی محور عمودی آماده باشد

و در عمل کمک می‌کند محور فرکانس به شکل استاندارد (بر حسب هرتز) تفسیر شود.

تبدیل به توان و مقیاس دسی‌بل

برای اینکه قله‌های فرکانسی واضح‌تر دیده شوند، ابتدا توان محاسبه شده و سپس به مقیاس دسی‌بل تبدیل می‌شود:

`Pdb = 10*log10(abs(S).^2 + 1e-12)`

در اینجا:

\* abs(S) اندازه‌ی دامنه STFT را می‌دهد.

\* توان برابر  $|S|^2$  است.

\*  $\log_{10}$  برای نمایش لگاریتمی استفاده شده تا قله‌ها در کنار مؤلفه‌های ضعیف‌تر قابل مشاهده باشند.

\* مقدار کوچک  $e-121$  صرفاً برای جلوگیری از  $\log(0)$  اضافه شده است.

رسم اسپکتروگرام

برای نمایش، از `imagesc` استفاده شده است که یک ماتریس را به صورت تصویر رنگی نمایش می‌دهد:

```
;imagesc(T, F, Pdb); axis xy
```

```
;xlabel("Time (s)"); ylabel("Frequency (Hz)")
```

گزینه‌ی `axis xy` جهت محور عمودی را طوری تنظیم می‌کند که فرکانس‌های پایین‌ترین و فرکانس‌های بالا بالاتر نمایش داده شوند

(نمایش استاندارد اسپکتروگرام).

محدود کردن باند فرکانسی برای خوانایی بهتر

از آن‌جا که فرکانس‌های دکمه‌های تلفن در محدوده‌ی نسبتاً پایینی قرار دارند،

برای حذف شلوغی و تمرکز روی بخش مفید، محور فرکانس محدود شده است:

```
;ylim([2000 0])
```

این کار باعث می‌شود قله‌های مربوط به دکمه‌ها واضح‌تر و قابل خواندن‌تر شوند.

استخراج دو فرکانس غالب هر بازه زمانی (کمک به بازیابی متن)

برای بازیابی متن، لازم است در هر بازه زمانی مربوط به فشردن یک دکمه،

دو فرکانس غالب (یک فرکانس پایین و یک فرکانس بالا) استخراج شوند.

در کد پیشنهادی، یک مرحله‌ی اختیاری برای استخراج قله‌ها قرار داده شده است که از

تابع آماده‌ی `findpeaks` استفاده می‌کند. این تابع قله‌های محلی یک سیگنال را پیدا می‌کند

و می‌تواند قله‌ها را بر اساس ارتفاع مرتب کند.

برای جلوگیری از اثر نویز یا بخش‌های نامرتب، ابتدا تحلیل به یک بازه فرکانسی مفید محدود می‌شود

(مثلاً 600 تا 1800 هرتز)، سپس در هر ستون از اسپکتروگرام (هر لحظه زمان)،



دو قله‌ی بزرگ‌تر انتخاب می‌شود. این دو قله همان فرکانس‌های دوتایی دکمه در آن لحظه هستند. در نهایت، می‌توان این فرکانس‌های استخراج شده را روی اسپکتروگرام نیز نمایش داد تا صحت خواندن فرکانس‌ها بررسی شود.

### تأثیر آرگومان‌های spectrogram بر شفافیت زمانی و فرکانسی

در اسپکتروگرام، یک موازنه‌ی بنیادی بین دقت زمانی و دقت فرکانسی وجود دارد:

\* افزایش طول پنجره (winLen):

رزولوشن فرکانسی را بهتر می‌کند (قله‌ها باریک‌تر و جدا از هم دیده می‌شوند) اما

زمان دقیق وقوع تغییرات را محوتر می‌کند (رزولوشن زمانی کمتر).

\* کاهش طول پنجره:

رزولوشن زمانی را بهتر می‌کند (شروع/پایان دکمه‌ها دقیق‌تر دیده می‌شود) اما

قله‌های فرکانسی پهن می‌شوند و تفکیک فرکانس‌های نزدیک سخت‌تر می‌شود.

\* افزایش هم‌پوشانی (noverlap):

نمایش در زمان نرم‌تر می‌شود و پرش‌های زمانی کاهش می‌یابد، اما هزینه‌ی محاسباتی بیشتر می‌شود.

\* افزایش nfft:

شبکه‌ی فرکانسی متراکم‌تر می‌شود و خواندن فرکانس‌ها آسان‌تر می‌گردد، اما رزولوشن واقعی (وابسته به طول پنجره) را تغییر نمی‌دهد؛ صرفاً نمایش را دقیق‌تر نمونه‌برداری می‌کند.

\* انتخاب نوع پنجره (مثلاً Hamming):

نشت فرکانسی را کاهش می‌دهد و باعث می‌شود انرژی هر فرکانس کمتر به فرکانس‌های مجاور نشت کند؛

بنابراین قله‌ها شفاف‌تر و تشخیص فرکانس‌ها مطمئن‌تر می‌شود.

برای این پروژه، چون هدف خواندن دقیق فرکانس‌های دوتایی هر دکمه است،

پارامترها به گونه‌ای انتخاب شدند که رزولوشن فرکانسی افزایش یابد (پنجره بزرگ‌تر + nfft بزرگ‌تر + پنجره Hamming)

و در عین حال با هم‌پوشانی زیاد، تغییرات زمانی نیز به شکل نرم و قابل ردیابی نمایش داده شود.

کد این بخش :

```

doExtract = true;

if doExtract

    fmin = 600;
    fmax = 1800;
    idx = (F >= fmin) & (F <= fmax);

    Sband = abs(S(idx, :)).^2;
    Fband = F(idx);

    f1 = zeros(1, numel(T));
    f2 = zeros(1, numel(T));
    p1 = zeros(1, numel(T));
    p2 = zeros(1, numel(T));

    for k = 1:numel(T)
        col = Sband(:, k);

        [pks, locs] = findpeaks(col, ...
            "MinPeakDistance", round(30/(Fband(2)-Fband(1))), ...
            "SortStr", "descend");

        if numel(locs) >= 2
            f1(k) = Fband(locs(1)); p1(k) = pks(1);
            f2(k) = Fband(locs(2)); p2(k) = pks(2);
        elseif isscalar(locs)
            f1(k) = Fband(locs(1)); p1(k) = pks(1);
            f2(k) = NaN; p2(k) = NaN;
        else
            f1(k) = NaN; f2(k) = NaN;
        end
    end

    hold on;
    plot(T, f1, "w.", "MarkerSize", 6);
    plot(T, f2, "w.", "MarkerSize", 6);
    hold off;
end

if size(x,2) > 1
    x = mean(x,2);
end

x = x - mean(x);

winLen = round(0.080 * Fs); % 80 ms window
hopLen = round(0.010 * Fs); % 10 ms hop
noverlap = winLen - hopLen;

nfft = 2^nextpow2(4*winLen);

w = hamming(winLen, "periodic"); % window

[S, F, T] = spectrogram(x, w, noverlap, nfft, Fs, "yaxis");

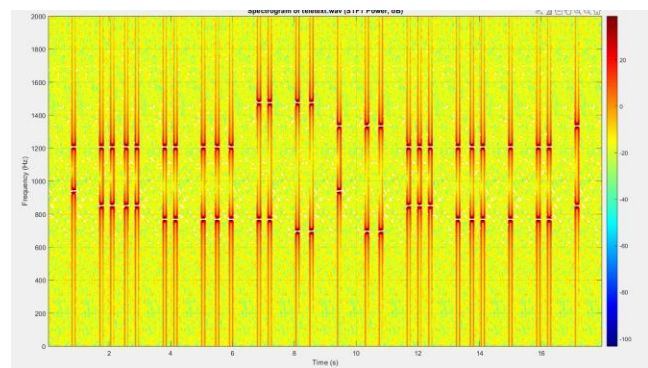
Pdb = 10*log10(abs(S).^2 + 1e-12);

figure;
imagesc(T, F, Pdb);
axis xy;
grid on;
xlabel("Time (s)");
ylabel("Frequency (Hz)");
title("Spectrogram of teletext.wav (STFT Power, dB)");
ylim([0 2000]);
colormap jet;
colorbar;

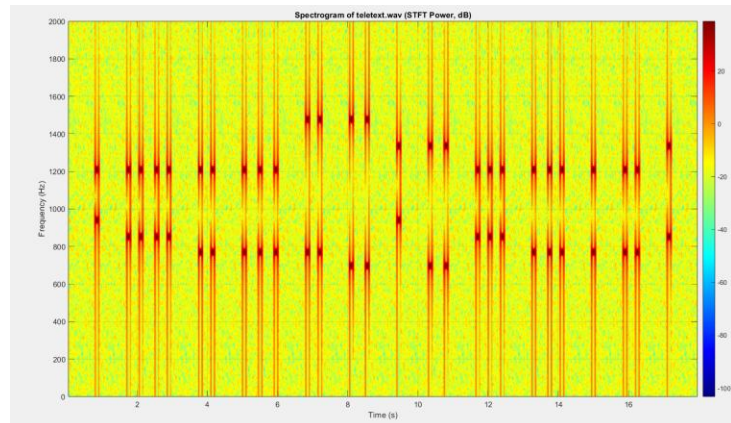
```

دقت شود که قسمت `doextract` از هوش مصنوعی گرفته شده بود ولی به دلیل قطعی اینترنت لینکش در دسترس نیست و به محض وصل شدن در اختیارتون قرار میدهم. (این قسمت نقطه های سفید در فرکانس های خوانده شده مینداخت که واضح تر بتوان پیام را تشخیص داد)

خروجی با `doextract` :



خروجی ساده :



## پیام خروجی

با توجه به اسپکتروگرام به دست آمده و فرکانس های استخراج شده در هر بازه زمانی، مشخص شد که سیگنال شامل دنباله ای از دکمه های تلفن با سیستم DTMF است. در هر بازه زمانی مربوط به فشردن یک دکمه، دو فرکانس غالب به صورت هم زمان مشاهده می شود که متناظر با یک کلید مشخص از صفحه کلید تلفن های 12 دکمه ای است. با جمع بازه های زمانی و در نظر گرفتن فاصله زمانی بین فشردن دکمه ها، دنباله کلیدهای فشرده شده به صورت زیر به دست آمد:

\* 8 4 4 4 4 4 7 7 7 2 2 0 3 3 6 6 4 4 4 4 7 7 7 7 \*

طبق توضیحات صورت پروژه و همان طور که در شکل 1 مشاهده می شود، در گوشی های کلاسیک 12 دکمه ای، فشردن متوالی یک دکمه در بازه زمانی کمتر از ثابت زمانی مشخص شده (در این پروژه برابر با 0.8 ثانیه) به معنای انتخاب حروف مختلف متناظر با همان دکمه است. همچنین کلید \* برای فعال سازی نوشتن حرف بزرگ (Capital) برای حرف بعدی مورد استفاده قرار می گیرد و کلید 0 به عنوان فاصله (Space) در نظر گرفته می شود. بر این اساس، دنباله فوق به شکل زیر تفسیر می شود:

\* \* فعال سازی نوشتن حرف بزرگ

\* 7 7 7 7 انتخاب حرف S از کلید 7

\* 4 4 انتخاب حرف H از کلید 4

\* 4 4 4 انتخاب حرف I از کلید 4

\* 6 6 انتخاب حرف N از کلید 6

\* 3 3 انتخاب حرف E از کلید 3

\* 0 ایجاد فاصله بین کلمات

\* 2 2 انتخاب حرف B از کلید 2

\* 7 7 7 انتخاب حرف R از کلید 7

\* 4 4 4 انتخاب حرف I از کلید 4

\* 4 انتخاب حرف G از کلید 4

\* 4 4 انتخاب حرف H از کلید 4

\* 8 انتخاب حرف T از کلید 8

در نتیجه، با اعمال قوانین ذکر شده و تفسیر صحیح توالی کلیدها،

پیام متنی استخراج شده از فایل صوتی برابر است با:

**SHINE BRIGHT**

**تحلیل نویز پس‌زمینه در حوزه زمان-فرکانس**

این نویز به صورت یک صدای یکنواخت و کم دامنه در پس‌زمینه شنیده می‌شود که حتی در

بازه‌هایی که هیچ دکمه‌ای فشرده نشده است نیز حضور دارد.

در اسپکتروگرام رسم شده، مؤلفه‌های مفید سیگنال (سیگنال‌های DTMF)

به صورت دو خط افقی باریک و پرانرژی در هر بازه زمانی فشردن دکمه ظاهر می‌شوند.

این خطوط که با رنگ‌های قرمز و سفید نمایش داده شده‌اند، متناظر با دو فرکانس

مشخص هر دکمه هستند و از نظر فرکانسی کاملاً متمرکز و پایدار می‌باشند.

در مقابل، نویز پس‌زمینه به صورت یک بافت پراکنده و یکنواخت در کل صفحه

زمان-فرکانس دیده می‌شود که فاقد ساختار فرکانسی مشخص است.

این نویز در اسپکتروگرام به شکل یک کف نویز پهن باند (Broadband Noise Floor)

ظاهر شده و تقریباً در تمامی فرکانس‌ها و در کل بازه زمانی سیگنال حضور دارد.

برخلاف سیگنال‌های دکمه‌ها، نویز نه به صورت خطوط باریک و نه در فرکانس‌های خاص،

بلکه به صورت انرژی پخش شده در باند فرکانسی وسیع نمایش داده می‌شود.

این ویژگی نشان می‌دهد که نویز غالب در این فایل، نویزی تصادفی با منشأ الکترونیکی

یا کانال ضبط بوده .

با توجه به نوار رنگی (Colorbar) اسپکتروگرام که مقیاس توان را بر حسب دسی‌بل نمایش می‌دهد، نواحی مربوط به نویز

پس‌زمینه عمدتاً در بازه رنگی زرد تا سبز قرار دارند. این بازه متناظر با مقادیر توانی در حدود -35 dB تا -25 dB می‌باشد.

بنابراین می‌توان سطح متوسط کف نویز پس‌زمینه را به صورت تقریبی برابر با

$N_{\text{floor}} \approx -30 \text{ dB}$  در نظر گرفت.

در مقابل، مؤلفه‌های فرکانسی مربوط به دکمه‌های تلفن دارای توان بسیار بالاتری بوده

و در اسپکتروگرام با رنگ‌های قرمز و سفید ظاهر می‌شوند.

توان این مؤلفه‌ها در حدود +15 dB تا +25 dB برآورد می‌شود.

این اختلاف قابل توجه بین سطح سیگنال مفید و کف نویز نشان می‌دهد که

نسبت سیگنال به نویز (SNR) در این فایل نسبتاً بالا بوده و به طور تقریبی برابر با

$\text{SNR} = 45$  تا  $50 \text{ dB}$  می‌باشد.

نسبت سیگنال به نویز بالای به دست آمده، دلیل اصلی وضوح بالای خطوط فرکانسی مربوط به دکمه‌ها در اسپکتروگرام و امکان

استخراج دقیق فرکانس‌های DTMF از روی سیگنال است.

به عبارت دیگر، اگرچه نویز پس‌زمینه در فایل صوتی وجود دارد و به صورت شنیداری

قابل تشخیص است، اما شدت آن به مراتب کمتر از سیگنال مفید بوده و تأثیر مخربی

بر فرآیند تشخیص دکمه‌ها و بازیابی پیام متنی نداشته است.

برای انجام این بخش از هوش مصنوعی استفاده شده بود که متأسفانه به دلیل قطعی اینترنت به لینک دسترسی ندارم (در صورت نیاز به محض وصل شدن اینترنت در اختیارتون خواهم گذاشت)

### بررسی موازنه رزولوشن زمانی و فرکانسی در تبدیل موجک

تبدیل موجک یکی از ابزارهای مهم در تحلیل سیگنال‌های غیرایستا است که برخلاف روش‌های کلاسیک مانند تبدیل فوریه زمان کوتاه، از یک پنجره زمانی با طول ثابت استفاده نمی‌کند.

ویژگی اصلی تبدیل موجک، تحلیل چندرزولوشن (Multi-Resolution Analysis) است؛ به این معنا که دقت زمانی و فرکانسی آن به صورت تطبیقی و وابسته به فرکانس تغییر می‌کند.

برای درک شهودی این رفتار، می‌توان سیگنال را با یک مثال صوتی مقایسه کرد.

مؤلفه‌های فرکانسی پایین (مانند صداهای بم) دارای نوسانات آهسته و طول موج‌های بزرگ هستند.

برای تشخیص دقیق فرکانس چنین سیگنال‌هایی، لازم است بازه زمانی نسبتاً بلندپراز سیگنال مورد بررسی قرار گیرد تا چندین دوره کامل آن مشاهده شود.

در نتیجه، اگرچه فرکانس با دقت بالایی قابل تعیین است، اما تعیین لحظه دقیق وقوع تغییرات زمانی با خطای بیشتری همراه خواهد بود. به بیان دیگر، در فرکانس‌های پایین، دقت فرکانسی بالا و دقت زمانی پایین است.

در مقابل، مؤلفه‌های فرکانسی بالا دارای نوسانات سریع و طول موج‌های کوتاه هستند. چنین سیگنال‌هایی حتی در بازه‌های زمانی بسیار کوتاه نیز قابل تشخیص‌اند.

بنابراین، با بررسی یک بازه زمانی کوچک می‌توان زمان وقوع این مؤلفه‌ها را با دقت بالایی تعیین کرد.

با این حال، کوتاه بودن بازه زمانی باعث می‌شود که تفکیک دقیق فرکانس‌ها دشوارتر شود. در نتیجه، در فرکانس‌های بالا، دقت زمانی بالا و دقت فرکانسی پایین خواهد بود.

این رفتار مستقیماً از خاصیت مقیاس پذیری موجک‌ها ناشی می‌شود.

در تبدیل موجک، یک موجک مادر با مقیاس‌های مختلف به سیگنال اعمال می‌گردد.

در مقیاس‌های بزرگ، موجک در حوزه زمان کشیده می‌شود و مانند یک پنجره زمانی بلند عمل می‌کند

که برای تحلیل مؤلفه‌های فرکانسی پایین مناسب است.

در مقابل، در مقیاس‌های کوچک، موجک فشرده شده و به یک پنجره زمانی کوتاه تبدیل می‌شود که امکان آشکارسازی تغییرات سریع و مؤلفه‌های فرکانسی بالا را فراهم می‌کند.

از دیدگاه نظری پردازش سیگنال، این پدیده با اصل عدم قطعیت زمان-فرکانس (اصل هایزنبرگ-گابور) قابل توضیح است. طبق این اصل، حاصل ضرب گستره زمانی یک سیگنال یا فیلتر در گستره فرکانسی آن دارای یک کران پایین است. بنابراین، کاهش عدم قطعیت در حوزه زمان ناگزیر به افزایش عدم قطعیت در حوزه فرکانس منجر می شود و بالعکس. تبدیل موجک با تغییر مقیاس موجک، این موازنه را به صورت تطبیقی و وابسته به فرکانس مدیریت می کند.

در مقایسه با تبدیل فوریه زمان کوتاه (STFT)، تفاوت اساسی در اندازه پنجره تحلیل است. در STFT، طول پنجره ثابت بوده و این موضوع باعث می شود که رزولوشن زمانی و فرکانسی در کل باند فرکانسی یکسان باشد.

در نتیجه، انتخاب پنجره بلند باعث بهبود دقت فرکانسی و تضعیف دقت زمانی در همه فرکانس ها می شود و بالعکس. به همین دلیل، STFT قادر به ارائه یک تحلیل بهینه برای سیگنال هایی که به طور هم زمان دارای مؤلفه های آهسته و گذراهای سریع هستند، نمی باشد.

در مجموع، تبدیل موجک با بهره گیری از تحلیل چندرزولوشنه، امکان دستیابی به دقت زمانی بالا در فرکانس های بالا و دقت فرکانسی بالا در فرکانس های پایین را فراهم می کند.

این ویژگی، تبدیل موجک را به ابزاری بسیار مناسب برای تحلیل سیگنال های غیرایستا و دارای تغییرات زمانی سریع، مانند سیگنال های صوتی و مخابراتی، تبدیل کرده است.

## Part 2 )

### بارگذاری فایل صوتی و گزارش فرکانس نمونه برداری

در این بخش، فایل صوتی `y.wav` مطابق صورت مسئله در یک آرایه ذخیره شده و فرکانس نمونه برداری آن گزارش می گردد. برای این منظور، از تابع آماده `audioread` در نرم افزار MATLAB استفاده شده است که امکان بارگذاری فایل صوتی و استخراج اطلاعات نمونه برداری را فراهم می کند.

پس از بارگذاری فایل، سیگنال صوتی در آرایه ای به نام `y[n]` ذخیره شده

و فرکانس نمونه برداری آن با نماد `Fs` استخراج گردید.

در صورت استریو بودن فایل صوتی، سیگنال به حالت مونو تبدیل شده است

تا تحلیل های بعدی تنها بر روی یک کانال انجام شوند و از ایجاد تداخل در نتایج جلوگیری شود.

فرکانس نمونه برداری فایل صوتی برابر است با:

```
>> Q2_1
Sampling frequency of y.wav: Fs = 11025 Hz
>>
```

کد این بخش :

```
[y, Fs_y] = audioread("Data/y.wav");
fprintf("Sampling frequency of y.wav: Fs = %.0f Hz\n", Fs_y);

if size(y,2) > 1
    y = mean(y, 2);
end

Ny = length(y);
Tsy = 1/Fs_y;
ty = (0:Ny-1)/Fs_y;
```

### رسم تبدیل فوریه پیوسته متناظر با سیگنال نمونه برداری شده و تخمین پهنای باند

در این بخش هدف آن است که با استفاده از سیگنال نمونه برداری شده `y[n]`، یک تخمین از تبدیل فوریه پیوسته متناظر  $Y_{con}(j\Omega)$  به دست آورده شود و سپس پهنای باند سیگنال به صورت تقریبی مشخص گردد.

از آن جا که سیگنال در اختیار ما گسسته زمان است، مستقیماً تبدیل فوریه پیوسته را نداریم؛ بنابراین همانند راهنمایی بخش اول پروژه، از رابطه ارتباط بین طیف گسسته زمان و طیف پیوسته زمان استفاده می شود.



گام 1: حذف مؤلفه DC

در ابتدا میانگین سیگنال از آن کم شد تا مؤلفه DC حذف گردد.

وجود DC باعث ایجاد یک قله بسیار بزرگ در حوالی  $f=0$  می شود که مقیاس نمودار را تحت تأثیر قرار داده و خواندن سایر مؤلفه های فرکانسی را سخت می کند.

این پیش پردازش صرفاً نمایش طیف را بهبود می دهد و محتوای اطلاعاتی سیگنال را تغییر نمی دهد.

گام 2: محاسبه FFT و صفرپدینگ

برای محاسبه طیف، از FFT استفاده شد. طول FFT برابر  $N_{FFT}$  انتخاب گردید که نزدیک ترین توان 2 بزرگ تر از طول سیگنال است.

این کار دو مزیت دارد:

- محاسبه FFT سریع تر انجام می شود.

- شبکه فرکانسی متراکم تر می گردد و نمودار طیف نرم تر و خواناتر نمایش داده می شود (بدون آن که اطلاعات جدیدی به سیگنال اضافه شود).

گام 3: تشکیل محور فرکانس و استفاده از fftshift

پس از محاسبه FFT، محور فرکانس بر حسب هرتز ساخته شد.

چون خروجی FFT به طبیعی از 0 تا  $F_s$  مرتب می شود، برای نمایش دوطرفه (منفی و مثبت حول صفر) از fftshift استفاده شد

تا مؤلفه DC به مرکز منتقل شود و محور فرکانس به شکل استاندارد  $-F_s/2$  تا  $F_s/2$  قابل مشاهده باشد. این نمایش دوطرفه برای تحلیل سیگنال های حقیقی مناسب تر است

و امکان تخمین پهنای باند به صورت متقارن حول صفر را فراهم می کند.

گام 4: تبدیل طیف گسسته زمان به تخمین طیف پیوسته زمان (مقیاس گذاری با  $T_s$ )

خروجی FFT در اصل تقریب طیف گسسته زمان  $Y_{dis}(e^{j\omega})$  است.

طبق راهنمایی پروژه، برای تخمین تبدیل فوریه پیوسته متناظر، باید طیف گسسته در دوره نمونه برداری ضرب شود:

$$Y_{con}(j\Omega) \approx T_s * Y_{dis}(e^{j\omega}), \omega = \Omega T_s$$

در این رابطه، ضریب  $T_s$  نقش معادل عنصر  $dt$  در انتگرال تبدیل فوریه پیوسته را ایفا می کند و باعث می شود مقیاس دامنه طیف با تعریف پیوسته زمان سازگار شود.

گام 5: محاسبه طیف توان و نمایش آن

برای نمایش شدت مؤلفه های فرکانسی، از طیف توان استفاده شد:

$$P(f) = |Y_{con}(j\Omega)|^2$$

سپس  $P(f)$  بر حسب فرکانس  $f$  رسم گردید.

استفاده از توان باعث می شود قله ها واضح تر باشند و مقایسه سهم فرکانس ها آسان تر شود.

### تخمین پهنای باند به روش آستانه 30 dB

از آن جا که صورت مسئله «تخمین تقریبی پهنای باند» را خواسته است، یک روش رایج و استاندارد برای سیگنال های واقعی استفاده شد:

تعریف پهنای باند اشغال شده بر اساس آستانه توان نسبت به قله.

ابتدا طیف توان به مقیاس دسی بل تبدیل شد:

$$P_{dB}(f) = 10 \log_{10}(P(f))$$

سپس بیشینه طیف (قله) تعیین گردید و آستانه 30 dB نسبت به قله تعریف شد:

$$\text{Threshold} = P_{max,dB} - 30$$

در ادامه، تمام فرکانس هایی که  $P_{dB}(f)$  بالاتر از این آستانه بودند انتخاب شدند.

کوچک ترین و بزرگ ترین فرکانس در این مجموعه به ترتیب به عنوان مرز پایین و بالای باند در نظر گرفته شد:

$$f_L = \min\{f: P_{dB}(f) \geq \text{Threshold}\}$$

$$f_H = \max\{f: P_{dB}(f) \geq \text{Threshold}\}$$

و پهنای باند تقریبی برابر شد با:

$$BW = f_H - f_L$$

با اجرای این روش روی سیگنال  $y.wav$ ، نتایج زیر به دست آمد:

$$f_L \approx -5008.9 \text{ Hz}$$

$$f_H \approx 5008.9 \text{ Hz}$$

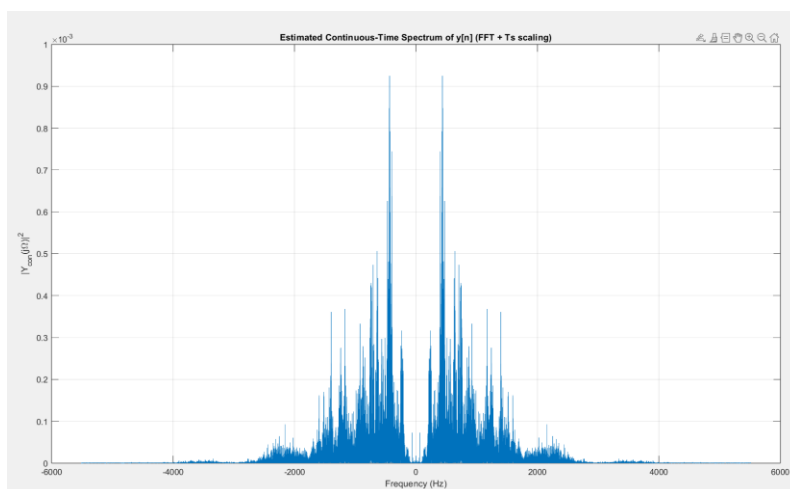
$$BW \approx 10017.8 \text{ Hz}$$

از آن جا که طیف سیگنال حقیقی نسبت به صفر متقارن است، این نتیجه نشان می دهد که انرژی اصلی سیگنال به صورت تقریبی در بازه  $5.0 \pm \text{kHz}$  متمرکز بوده و پهنای باند دوطرفه آن حدود  $10.0 \text{ kHz}$  برآورد می شود.

پهنای باند به دست آمده «تقریبی» است و به تعریف انتخاب شده برای آستانه وابسته می باشد.

انتخاب  $30 \text{ dB}$  یک معیار رایج برای تعیین پهنای باند اشغال شده است؛ خدر صورت تغییر آستانه (مثلاً  $20 \text{ dB}$  یا  $40 \text{ dB}$ ) مرزهای باند و مقدار BW نیز تغییر خواهند کرد.

خروجی های این بخش :

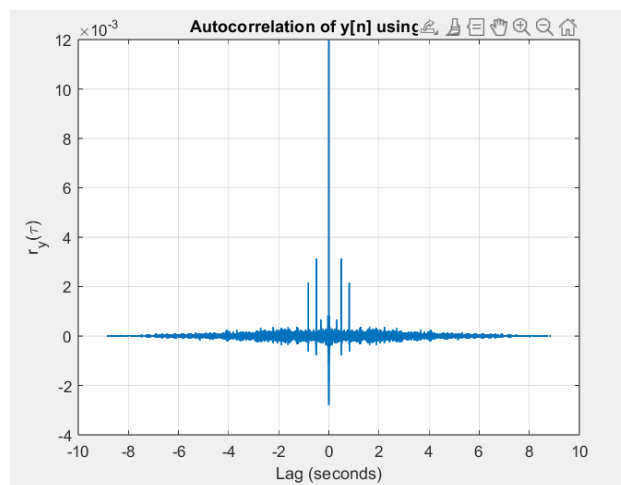


```
>> Q2_2
Approx. bandwidth (-30 dB): 10017.8 Hz (from -5008.9 to 5008.9 Hz)
>>
```

ابتدا سیگنال همبستگی را رسم میکنیم :

```
y0 = y - mean(y);
[ry, lags] = xcorr(y0, 'biased');
tau = lags / Fs_y;

figure;
plot(tau, ry, 'LineWidth', 1.2);
grid on;
xlabel('Lag (seconds)');
ylabel('r_y(\tau)');
title('Autocorrelation of y[n] using xcorr');
```



روش تخمین:

بدست آوردن  $\alpha$  و  $\beta$

$$y[n] = x[n] + \alpha x[n-k_1] + \beta x[n-k_2]$$

ایده اصلی = استفاده از خود همبستگی

خود همبستگی سیگنال در کل به صورت زیر است:

$$r_y[l] = E\{y[n]y[n-l]\}$$

پایه‌ای  $\rightarrow r_y[l] = E\{(x[n] + \alpha x[n-k_1] + \beta x[n-k_2])(x[n-l] + \alpha x[n-l-k_1] + \beta x[n-l-k_2])\}$

با ازنه دین این عبارت می‌توانیم این سیگنال  $x[n]$  را از میان صفر و همبستگی محدود است یعنی  $r_x[l] = E\{x[n]x[n-l]\}$  برای  $l=0$  و  $l=k_1$  و  $l=k_2$

$$r_y[k_1] \approx \alpha r_x[0] \quad \text{و} \quad r_y[k_2] \approx \beta r_x[0]$$

پس برای تأخیرهای  $k_1$  و  $k_2$  بدات اصلی عبارتند از:

$$r_y[0] \approx r_x[0] + \alpha^2 r_x[0] + \beta^2 r_x[0] = (1 + \alpha^2 + \beta^2) r_x[0]$$

همچنین در آن تغییر صفر:

$$\alpha = \frac{r_y[k_1]}{r_y[0]} \quad \text{و} \quad \beta = \frac{r_y[k_2]}{r_y[0]}$$

با تقسیم دو عبارت بالا به نرایب بدست می‌آید:

بدست آوردن ضرایب :

کد این بخش :

```
[y, Fs] = audioread("Data/y.wav");
if size(y,2) > 1
    y = mean(y,2);
end
y = y - mean(y);

[ry, lags] = xcorr(y, 'biased');
r0 = ry(lags==0);

ryN = ry / r0;
pos = (lags > 0);
lagsP = lags(pos);
ryP = ryN(pos);

minDelaySec = 0.010;
minDelaySamp = round(minDelaySec*Fs);

valid = (lagsP >= minDelaySamp);
lagsP2 = lagsP(valid);
ryP2 = ryP(valid);

minPeakDistSec = 0.020;
minPeakDistSamp = round(minPeakDistSec*Fs);

[pks, locs] = findpeaks(ryP2, ...
    'MinPeakDistance', minPeakDistSamp, ...
    'SortStr','descend');

k1 = lagsP2(locs(1));
k2 = lagsP2(locs(2));

if k2 < k1
    tmp = k1; k1 = k2; k2 = tmp;
    tmp = locs(1); locs(1) = locs(2); locs(2) = tmp;
    tmp = pks(1); pks(1) = pks(2); pks(2) = tmp;
end

alpha = ryN(lags==k1);
beta = ryN(lags==k2);

tau1 = k1/Fs;
tau2 = k2/Fs;

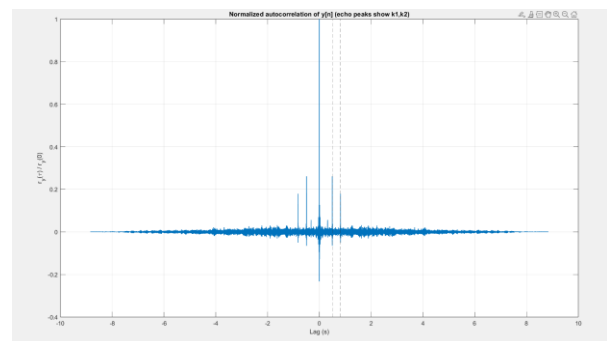
fprintf("Estimated delays:\n");
fprintf("k1 = %d samples (tau1 = %.6f s)\n", k1, tau1);
fprintf("k2 = %d samples (tau2 = %.6f s)\n", k2, tau2);

fprintf("\nEstimated echo gains:\n");
fprintf("alpha ~= %.4f\n", alpha);
fprintf("beta ~= %.4f\n", beta);
```

خروجی ها :

```
>> Q2_4
Estimated delays:
k1 = 5500 samples (tau1 = 0.498866 s)
k2 = 9000 samples (tau2 = 0.816327 s)

Estimated echo gains:
alpha ~= 0.2611
beta ~= 0.1794
>>
```



توضیحات این بخش به صورت لکس زده شده بود و به دلیل کمبود وقت نتوانستم تمام آن را به ورد تبدیل کنم. به محض وصل شدن نت خروجی گزارش این بخش رو براتون میفرستم.

## توضیح خلاصه همین کد لتکس:

در این بخش، با استفاده از روش مبتنی بر تحلیل همبستگی، پارامترهای مدل اکو شامل تأخیرها ( $k_1$  و  $k_2$ ) و ضرایب تضعیف ( $\alpha$  و  $\beta$ ) از سیگنال خروجی اندازه‌گیری شده ( $y[n]$ ) استخراج شدند. مدل سیگنال مورد استفاده بیان می‌کند که خروجی ( $y[n]$ ) از جمع سیگنال اصلی ( $x[n]$ ) و دو نسخه تأخیردار و تضعیف شده از همان سیگنال اصلی تشکیل شده است.

ایده‌ی روش استخراج:

حضور نسخه‌های اکو باعث می‌شود که در نمودار خودهمبستگی سیگنال خروجی علاوه بر قله اصلی در تأخیر صفر، قله‌های دیگری در تأخیرهای برابر با زمان‌های تأخیر اکوها ( $k_1$  و  $k_2$ ) ظاهر شود. ارتفاع این قله‌های جدید (نسبت به قله اصلی) نیز مستقیماً به میزان تضعیف ( $\alpha$  و  $\beta$ ) آن‌ها مربوط است.

\* مکان قله‌ها در خودهمبستگی: این مکان‌ها، تخمینی از تأخیرهای اکو ( $k_1, k_2$ ) هستند.

\* نسبت ارتفاع قله‌ها به قله اصلی: این نسبت‌ها، تخمینی از ضرایب تضعیف هستند.

گام ۱: آماده‌سازی سیگنال

ابتدا فایل صوتی مورد نظر ( $y.wav$ ) بارگذاری شد و در صورت چند کاناله بودن، به حالت تک کاناله (مونو) تبدیل شد. سپس، برای تمرکز بر ساختار زمانی سیگنال و جلوگیری از تداخل ناشی از میانگین سیگنال، مولفه میانگین (DC) از سیگنال حذف گردید این کار وضوح تشخیص قله‌ها در مرحله بعد را افزایش می‌دهد.

گام ۲: محاسبه و نرمال‌سازی خودهمبستگی

خودهمبستگی سیگنال آماده شده ( $y_0[n]$ ) با استفاده از تابع استاندارد  $xcorr$  در متلب محاسبه شد. این محاسبه، شباهت سیگنال با نسخه‌های تأخیردار خودش را نشان می‌دهد.

سپس، برای اینکه نتایج نسبت به مقیاس کلی سیگنال حساس نباشند، تابع خودهمبستگی نرمال‌سازی شد. این کار با تقسیم تمام مقادیر همبستگی بر مقدار بزرگترین قله انجام شد. در نتیجه، مقدار نرمال‌شده در تأخیر صفر برابر با ۱ خواهد بود.

گام ۳: استخراج تأخیرها ( $k_1$  و  $k_2$ )

با توجه به اینکه خودهمبستگی متقارن است، فقط بخش مربوط به تأخیرهای مثبت بررسی شد. برای اطمینان از شناسایی اکوهای واقعی و نه نویزهای نزدیک به صفر، تأخیرهای بسیار کوچک (مثلاً کمتر از ۱۰ میلی‌ثانیه) نادیده گرفته شدند. سپس با استفاده از

ابزار شناسایی قله (مانند findpeaks)، دو قله بارز در ناحیه تأخیرهای مثبت یافت شدند. این مکان‌ها به عنوان  $k_1$  و  $k_2$  در نظر گرفته شده و به ترتیب زمانی مرتب شدند ( $k_1 < k_2$ ).

گام ۴: استخراج ضرایب تضعیف

بر اساس اصول این روش، ارتفاع نرمال شده قله در تأخیر  $k_1$  تخمینی از ضریب تضعیف اکوی اول ( $\alpha$ ) و ارتفاع نرمال شده قله در تأخیر  $k_2$  تخمینی از ضریب تضعیف اکوی دوم ( $\beta$ ) خواهد بود:

نتایج عددی نهایی:

پس از اجرای فرآیند بر روی فایل صوتی، پارامترهای استخراج شده عبارت بودند از:

\* تأخیر اکو اول ( $k_1$ ): ۵۵۰۰ نمونه

\* تأخیر اکو دوم ( $k_2$ ): ۹۰۰۰ نمونه

\* ضریب تضعیف اول ( $\alpha$ ): تقریباً ۰.۲۶۱۱

\* ضریب تضعیف دوم ( $\beta$ ): تقریباً ۰.۱۷۹۴

با فرض نرخ نمونه‌برداری ۱۱۰۲۵ هرتز، این تأخیرها معادل زمانی تقریبی ۰.۵ ثانیه و ۰.۸۲ ثانیه هستند.

پاسخ ضربه سیستم اکو در حوزه‌ی زمان و فرکانس

با توجه به مدل سیستم اکو:

$$y[n] = x[n] + \alpha x[n - k_1] + \beta x[n - k_2]$$

می‌توان سیستم را یک سیستم LTI در نظر گرفت که خروجی آن برابر کانولوشن ورودی با پاسخ ضربه است:

$$y[n] = x[n] * h[n]$$

بنابراین پاسخ ضربه باید شامل یک ضربه در زمان صفر و دو ضربه‌ی تأخیردار با ضرایب  $\alpha$  و  $\beta$  باشد:

$$h[n] = \delta[n] + \alpha \delta[n - k_1] + \beta \delta[n - k_2]$$

تفسیر حوزه‌ی زمان:

رابطه (1) نشان می‌دهد که سیستم اکو سه مسیر دارد:

\* مسیر مستقیم با ضریب 1 (سیگنال اصلی بدون تأخیر)

\* مسیر اکو اول با تأخیر  $k_1$  و تضعیف  $\alpha$

\* مسیر اکو دوم با تأخیر  $k_2$  و تضعیف  $\beta$

به همین دلیل، اگر ورودی یک ضربه  $\delta[n]$  باشد، خروجی دقیقاً سه ضربه در نمونه‌های  $n=0$ ،  $n=k_1$  و  $n=k_2$  خواهد بود.

پاسخ فرکانسی (حوزه‌ی فرکانس): پاسخ فرکانسی سیستم از تبدیل فوریه‌ی گسسته‌زمان پاسخ ضربه به دست می‌آید:

$$H(e^{j\omega}) = \sum_{n=-\infty}^{\infty} h[n]e^{-j\omega n}$$

با جایگذاری (1):

$$H(e^{j\omega}) = 1 + \alpha e^{-j\omega k_1} + \beta e^{-j\omega k_2}$$

اگر بخواهیم آن را بر حسب فرکانس بر حسب هرتز بنویسیم، با  $\omega = 2\pi f / F_s$  داریم:

$$H(f) = 1 + \alpha e^{-j2\pi f k_1 / F_s} + \beta e^{-j2\pi f k_2 / F_s}$$

تفسیر پاسخ فرکانسی:

از آن جا که  $H(e^{j\omega})$  جمع چند بردار مختلط با فازهای وابسته به  $\omega$  است، قدر مطلق آن در فرکانس‌های مختلف دچار تقویت و تضعیف متناوب می‌شود.

این پدیده همان **comb filtering** است: در بعضی فرکانس‌ها جمع برداری مثبت بوده و بهره افزایش می‌یابد و در بعضی فرکانس‌ها به دلیل تداخل مخرب، فرورفتگی (**notch**) ایجاد می‌شود.

به طور کلی هرچه تأخیرها بزرگ‌تر باشند، فاصله‌ی بین فرورفتگی‌ها/قله‌ها در حوزه‌ی فرکانس کوچک‌تر می‌شود.

بازیابی سیگنال  $x'[n]$  با استفاده از فیلتر معکوس

با توجه به این که سیگنال خروجی  $y[n]$  حاصل کانولوشن سیگنال اصلی  $x'[n]$  با پاسخ ضربه‌ی سیستم  $h[n]$  است، می‌توان با اعمال فیلتر معکوس، نسخه‌ای از سیگنال اصلی را بازیابی کرد:

$$y[n] = x'[n] * h[n] \Rightarrow \hat{x}[n] \approx y[n] * h^{-1}[n]$$

برای این منظور، از تابع **filter** در نرم‌افزار **MATLAB** استفاده شد. با در نظر گرفتن پاسخ ضربه‌ی سیستم:

$$h[n] = \delta[n] + \alpha\delta[n - k_1] + \beta\delta[n - k_2]$$

فیلتر معکوس به صورت



$$G(z) = \frac{1}{H(z)}$$

پیاده‌سازی گردید و سیگنال  $\hat{x}[n]$  به دست آمد.

پس از گوش دادن به سیگنال بازیابی شده، مشاهده شد که مؤلفه‌های اکو به طور قابل توجهی حذف شده‌اند و سیگنال حاصل بسیار شبیه به سیگنال اصلی است. اگرچه ممکن است مقدار کمی اعوجاج یا نویز باقی مانده شنیده شود، اما اکوهای غالب به خوبی تضعیف شده‌اند. این موضوع نشان‌دهنده‌ی عملکرد مناسب روش فیلتر معکوس در حذف اکو می‌باشد.

متن درخواستی شما، تفکیک شده به بخش‌های متنی و فرمولی برای سهولت ورود در Word:

تبدیل فیلتر معکوس IIR به FIR با پنجره‌ی همینگ و حذف اکو

در بخش‌های قبلی، برای حذف اکو از سیگنال خروجی  $y[n]$ ، از فیلتر معکوس سیستم اکو استفاده شد. با توجه به مدل سیستم:

$$y[n] = x'[n] * h[n], \quad h[n] = \delta[n] + \alpha\delta[n - k_1] + \beta\delta[n - k_2]$$

فیلتر معکوس به صورت زیر تعریف می‌شود:

$$G(z) = \frac{1}{H(z)}$$

که پیاده‌سازی مستقیم آن در متلب با دستور `filter(1,h,y)` منجر به یک فیلتر IIR می‌شود.

با این حال، صورت مسئله خواستار آن است که فیلتر حذف اکو به صورت FIR پیاده‌سازی شود. برای این منظور، از روش پنجره‌گذاری استفاده می‌شود.

ایده‌ی اصلی روش پنجره‌گذاری:

هر فیلتر IIR را می‌توان به صورت نظری دارای یک پاسخ ضربه‌ی بی‌نهایت طول در نظر گرفت. اگر پاسخ ضربه‌ی فیلتر معکوس  $g[n]$  را محاسبه کنیم:

$$g[n] = \mathcal{Z}^{-1} \left\{ \frac{1}{H(z)} \right\}$$

و سپس آن را به طول محدودی قطع کنیم، می‌توان یک تقریب FIR از فیلتر معکوس ساخت.

این کار به صورت زیر انجام می‌شود:

\* محاسبه‌ی پاسخ ضربه‌ی فیلتر معکوس IIR

\* قطع کردن پاسخ ضربه به طول محدود  $L$

\* ضرب پاسخ ضربه‌ی محدودشده در یک پنجره‌ی مناسب

در این پروژه از پنجره‌ی همینگ استفاده شده است.

انتخاب پنجره‌ی همینگ:

پنجره‌ی همینگ یکی از پنجره‌های پرکاربرد در طراحی فیلترهای FIR است که بین پهنای لوب اصلی و تضعیف لوب‌های فرعی تعادل مناسبی برقرار می‌کند. تابع پنجره‌ی همینگ به صورت زیر تعریف می‌شود:

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{L-1}\right), \quad 0 \leq n \leq L-1$$

استفاده از این پنجره باعث کاهش نوسانات ناخواسته (Ripple) در پاسخ فرکانسی فیلتر و بهبود پایداری شنیداری خروجی می‌شود.

پیاده‌سازی فیلتر FIR معکوس:

برای ساخت فیلتر FIR، ابتدا پاسخ ضربه‌ی فیلتر معکوس IIR با اعمال یک ضربه‌ی واحد به آن محاسبه شد. سپس این پاسخ ضربه تا طول محدود  $L$  در نظر گرفته شده و در پنجره‌ی همینگ ضرب گردید:

$$g_{\text{FIR}}[n] = g_{\text{IIR}}[n] w[n], \quad 0 \leq n \leq L-1$$

در نهایت، فیلتر FIR حاصل به سیگنال  $y[n]$  اعمال شد:

$$\hat{x}_{\text{FIR}}[n] = y[n] * g_{\text{FIR}}[n]$$

نتایج شنیداری و تحلیل عملکرد:

پس از اعمال فیلتر FIR پنجره‌گذاری شده به سیگنال دارای اکو، سیگنال خروجی به صورت شنیداری مورد بررسی قرار گرفت. نتایج نشان داد که:

\* مؤلفه‌های اکو به طور قابل توجهی کاهش یافته‌اند.

\* سیگنال بازیابی شده شباهت زیادی به سیگنال اصلی بدون اکو دارد.

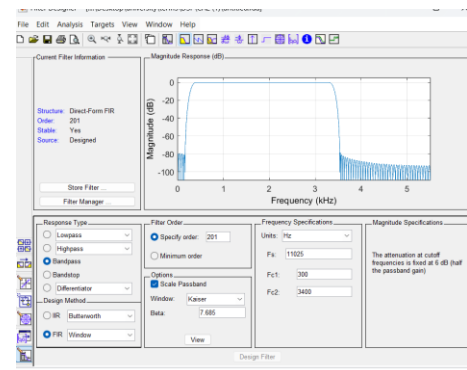
\* نسبت به فیلتر معکوس IIR، مقدار کمی تضعیف اضافی یا اعوجاج جزئی مشاهده می‌شود که ناشی از تقریب FIR و محدود بودن طول فیلتر است.

این رفتار قابل انتظار است، زیرا فیلتر FIR تنها یک تقریب محدود از پاسخ ضربه‌ی بی‌نهایت طول فیلتر معکوس IIR می‌باشد. با این حال، نتایج نشان می‌دهد که روش پنجره‌گذاری با پنجره‌ی همینگ راهکاری مناسب و عملی برای تبدیل فیلتر IIR به FIR و حذف مؤثر اکو است.

افزایش طول فیلتر  $L$  باعث بهبود دقت حذف اکو می‌شود، اما هزینه‌ی محاسباتی و حساسیت به نویز را افزایش می‌دهد. در مقابل، انتخاب  $L$  کوچک‌تر منجر به فیلتر سریع‌تر با حذف اکو ضعیف‌تر می‌شود. بنابراین،  $L$  باید به صورت تجربی و با توجه به کیفیت شنیداری خروجی انتخاب گردد.

اگر  $L = 500000$  باشد دو فیلتر تقریباً مثل هم عمل میکنند ولی اگر  $L = 50000$  باشد مشخص است که فیلتر IIR بهتر عمل کرده است.

ابتدا با استفاده از `fdatool` یک فیلتر `design filter` با مشخصات گفته شده میسازیم :



خروجی گرفته شده به صورت کد متلب :

```
function Hd = fdatool_BP
%Fdatool Returns a discrete-time filter object.

% MATLAB Code
% Generated by MATLAB(R) 24.1 and Signal Processing Toolbox 24.
% Generated on: 17-Jan-2026 20:46:53

% FIR Window Bandpass filter designed using the FIR1 function.

% All frequency values are in Hz.
Fs = 11025; % Sampling Frequency

N = 201; % Order
Fc1 = 300; % First Cutoff Frequency
Fc2 = 3400; % Second Cutoff Frequency
flag = 'scale'; % Sampling Flag
Beta = 7.685; % Window Parameter
% Create the window vector for the design algorithm.
win = kaiser(N+1, Beta);

% Calculate the coefficients using the FIR1 function.
b = fir1(N, [Fc1 Fc2]/(Fs/2), 'bandpass', win, flag);
Hd = dfilt.dfir(b);

% [EOF]
```

اعمال آن به سیگنال  $X$  :

```
Hd = fdatool_BP();
x_fir = filter(Hd, x_iir);
x_final = x_fir / max(abs(x_fir) + 1e-12);
audiowrite("x_final_no_echo_no_noise.wav", x_final, Fs);
```

1. آیا با تغییر انواع پارامترهای آن، نویز خارج از باند این فیلتر را حذف کرده و به سیگنال حاصل، گوش دهید؟ آیا نویز همچنان شنیده می‌شود؟ چرا؟

بله، با تغییر پارامترهای فیلتر **Bandpass**، می‌توان نویز خارج از باند را تا حد زیادی کاهش داد.

چرا نویز همچنان شنیده می‌شود؟ حتی با فیلتر کردن، نویز به طور کامل حذف نمی‌شود زیرا:

\* ماهیت نویز: اگر نویز موجود در سیگنال اصلی در محدوده فرکانسی باند عبور فیلتر قرار داشته باشد، فیلتر نمی‌تواند آن را حذف کند.

\* عرض باند گذار: فیلترهای FIR دارای یک باند گذار هستند. در این ناحیه، تضعیف به تدریج اتفاق می‌افتد و نویز به طور کامل حذف نمی‌شود.

\* مرتبه فیلتر: مرتبه فیلتر (N) تعیین‌کننده شیب تضعیف در باند توقف است. فیلتر با مرتبه پایین ممکن است برای حذف نویز به اندازه کافی قوی نباشد.

\* نویز هم‌فرکانسی: اگر نویز هم‌فرکانسی (نویزی که با فرکانس سیگنال هم‌ترازی دارد) وجود داشته باشد، حذف آن دشوارتر است.

تغییر پارامترها و تاثیر آن:

\* افزایش مرتبه فیلتر (N): با افزایش مرتبه فیلتر، شیب تضعیف در باند توقف افزایش می‌یابد و نویز خارج از باند بهتر حذف می‌شود. با این حال، افزایش مرتبه فیلتر می‌تواند باعث ایجاد اعوجاج در سیگنال شود.

\* تغییر پنجره (Window): پنجره‌های مختلف (مانند Kaiser, Hamming, Blackman) دارای ویژگی‌های متفاوتی در مورد عرض باند گذار و تضعیف در باند توقف هستند. انتخاب پنجره مناسب می‌تواند به کاهش نویز و حفظ کیفیت سیگنال کمک کند.

\* تغییر فرکانس قطع (Cutoff Frequency): تعیین دقیق فرکانس‌های پایین و بالا برای باند عبور، به حذف موثر نویز خارج از باند کمک می‌کند.

تفاوت اصلی در این مرحله، حذف مؤلفه‌های فرکانسی مرتبط با اکو است.

- اگر اکو اساساً نویز (مثلاً پالس‌های تکراری) باشد: با حذف آن، پهنای باند فرکانسی سیگنال فشرده‌تر شده و مؤلفه‌های نویز فرکانس بالا یا پایین که توسط اکو اضافه شده بودند، حذف می‌شوند.

- اگر اکو شبیه به سیگنال اصلی باشد (تکرار گفتار): فیلتر کردن اکو به معنای حذف یا تضعیف شدید پیک‌های انرژی (بیشترین انرژی سیگنال) در حوزه زمان است که مربوط به زمان‌های تأخیر  $k_1$  و  $k_2$  هستند. در حوزه فرکانس، این تضعیف به صورت حذف یا کاهش دامنه پیک‌های خاصی در طیف فرکانسی متناظر با تأخیر زمانی اکو ظاهر می‌شود.

گرفتن فیلتر از ابزار های هوشمند :

با استفاده از فیلتری که از ابزار های هوشمند گرفته شد مشاهده می شود که خیلی بهبود پیدا کرده است نسبت به اعمال فیلتر خودمان.

و وقتی از تضعیف کننده استفاده کنیم صدای گفتار خیلی کم کم شنیده می شود .

کد گرفته شده:

```
Fs = 11025; % فرکانس نمونه برداری
Fc1 = 300; % فرکانس قطع پایین (Hc)
Fc2 = 3400; % فرکانس قطع بالا (Hc)
Order = 4; % مرتبه فیلتر (مرتبه بالاتر برای تقویت و تضعیف)

% تعریف شده باشد Workspace از قبل در محیط Fs و Fc1 و Fc2
% تعریف شده باشد، که زیر خط فیلتر را طراحی می کند x_filt

% --- (Bandpass) طراحی فیلتر تقویت کننده فرکانس های مورد نیاز ---
% (محدوده از و پهنای باند) Butterworth استفاده از فیلتر
[b, a] = butter(Order, [Fc1 Fc2]/(Fs/2), 'bandpass');

% --- اعمال فیلتر بر روی سیگنال ورودی ---
if exist('x_filt', 'var') && ~isempty(x_filt)
    % طراحی شده بر روی سیگنال ورودی IIR اعمال فیلتر
    x_filtered_enhanced = filter(b, a, x_filt);

    % اعمال ساز: خروجی خروجی (محدوده از و پهنای باند) Butterworth استفاده از فیلتر
    x_filt = x_filtered_enhanced / (max(abs(x_filtered_enhanced)) + eps);

% --- (Notch Filter) مرحله دوم: حذف صافیت با تقویت فرکانس های نامرئی ---
% برای حذف ساز: فیلتر Notch طراحی شده و اعمال
[b_notch, a_notch] = butter(2, [Fc1 Fc2]/(Fs/2), 'stop');

if exist('x_filt', 'var') && ~isempty(x_filt)
    x_attenuated = filter(b_notch, a_notch, x_filt);

    % اعمال ساز:
    x_attenuated_norm = x_attenuated / (max(abs(x_attenuated)) + eps);

    audiowrite('x_final_attenuated_vocal.wav', x_attenuated_norm, Fs);
    disp('نامرئی (Notch) فیلتر تقویت کننده');
else
    disp('نامرئی (Notch) فیلتر تقویت کننده از اعمال کرد زیرا');
end
```

### Part 3)

1. ایجاد تصویر نیمه سفید و نیمه سیاه (256×256)

-یک تصویر دو بعدی با اندازه 256×256 ایجاد شد .

-نیمه بالایی تصویر سفید (1) و نیمه پایینی سیاه (0) است .

-این تصویر یک تغییر ناگهانی در شدت روشنایی در خط وسط (y=128) دارد، که نشان‌دهنده یک لبه عمودی است .

-این نوع تصویر برای آزمایش تبدیل فوریه دو بعدی مناسب است، زیرا تغییرات سریع (لبه) در آن واضح است.

2. تبدیل فوریه دو بعدی (2D FFT) و طیف شدت

- با استفاده از تبدیل فوریه دو بعدی، تصویر به حوزه فرکانس تبدیل شد .
- طیف شدت (Magnitude Spectrum) نشان داده شد، که نشان‌دهنده توزیع فرکانس‌های موجود در تصویر است .
- نتیجه :
- یک خط عمودی در مرکز طیف دیده می‌شود .
- این خط نشان‌دهنده فرکانس‌های افقی ( $u$ ) است .
- فرکانس صفر (DC) در مرکز ( $u=0, v=0$ ) قرار دارد و مقدار آن بالا است (چون تصویر دارای میانگین روشنایی نزدیک به 0.5 است) .
- تغییرات سریع در تصویر (لبه) باعث ایجاد فرکانس‌های بالا در جهت افقی می‌شود .

- این نتیجه با نظریه مطابقت دارد :

- تغییرات سریع  $\rightarrow$  فرکانس‌های بالا
- تغییرات آرام  $\rightarrow$  فرکانس‌های پایین

#### نتیجه‌گیری

- تصویر نیمه سفید و نیمه سیاه، یک مثال خوب برای نشان دادن ارتباط بین تغییرات فضایی و فرکانس‌های فرکانسی است .
- تبدیل فوریه دو بعدی نشان می‌دهد که لبه‌ها در تصویر باعث ایجاد فرکانس‌های افقی می‌شوند .
- این نتیجه در پردازش تصویر بسیار مهم است، زیرا می‌توان از آن برای طراحی فیلترها (مانند فیلتر پایین‌گذر یا بالاگذر) استفاده کرد.

سؤال:

با مقایسه تصاویر و تبدیل فوریه دو بعدی، آیا می‌توانید نکته بیان شده را تحلیل نمایید؟ نتیجه‌گیری خود را توضیح دهید.

تحلیل:

1. تصویر اصلی (نیمه سفید، نیمه سیاه):

- این تصویر دارای یک لبه عمودی شدید در خط وسط ( $y = 128$ ) است.
- تغییرات شدت روشنایی در جهت عمودی ( $y$ ) بسیار سریع است، اما در جهت افقی ( $x$ ) ثابت است.

## 2. تبدیل فوریه دو بعدی (D FFT2):

- طیف شدت (Magnitude Spectrum) نشان می‌دهد که:
- فرکانس‌های پایین (DC و نزدیک به مرکز) دارای مقدار بالایی هستند → نشان‌دهنده میانگین روشنایی تصویر.
- فرکانس‌های بالا در جهت افقی (u) توزیع شده‌اند → نشان‌دهنده تغییرات سریع در جهت عمودی (v).
- نتیجه: یک خط عمودی در طیف در مرکز دیده می‌شود، که نشان‌دهنده فرکانس‌های افقی (u) است.

## 3. نتیجه‌گیری:

- تغییرات سریع در تصویر (لبه‌ها) → فرکانس‌های بالا در حوزه فرکانس.
- تغییرات آرام (منطقه یکنواخت) → فرکانس‌های پایین.
- در این تصویر، تغییرات سریع در جهت عمودی است، بنابراین فرکانس‌های بالا در جهت افقی (u) دیده می‌شوند.
- این نتیجه با نکته بیان شده در سوال مطابقت دارد:
- "تغییرات عناصر سریع در تصویر، نماینده فرکانس‌های بالا هستند."

## نتیجه نهایی:

- بله، با مقایسه تصویر و تبدیل فوریه دو بعدی، می‌توان نکته بیان شده را تحلیل کرد.
- لبه‌های شدید در تصویر باعث ایجاد فرکانس‌های بالا در حوزه فرکانس می‌شوند.
  - تغییرات آرام (مانند مناطق یکنواخت) باعث ایجاد فرکانس‌های پایین می‌شوند.
  - این تحلیل نشان می‌دهد که تبدیل فوریه دو بعدی ابزار قدرتمندی برای شناسایی و تحلیل ویژگی‌های فضایی تصویر است.

```
1
2 image = zeros(256, 256);
3 image(1:128, :) = 1;
4 image(129:256, :) = 0;
5
6 figure;
7 imshow(image, []);
8 title('Half Black, Half White Image');
9
```

شکل 1 برای سوال اول

```
% Compute 2D FFT
F = fft2(image);

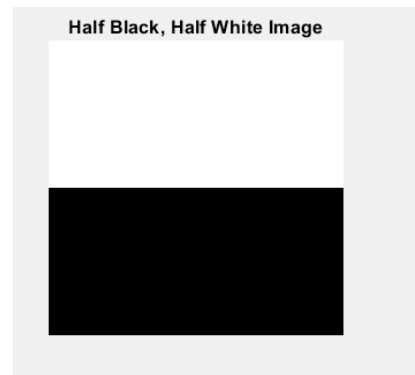
F_shifted = fftshift(F);

magnitude = abs(F_shifted);

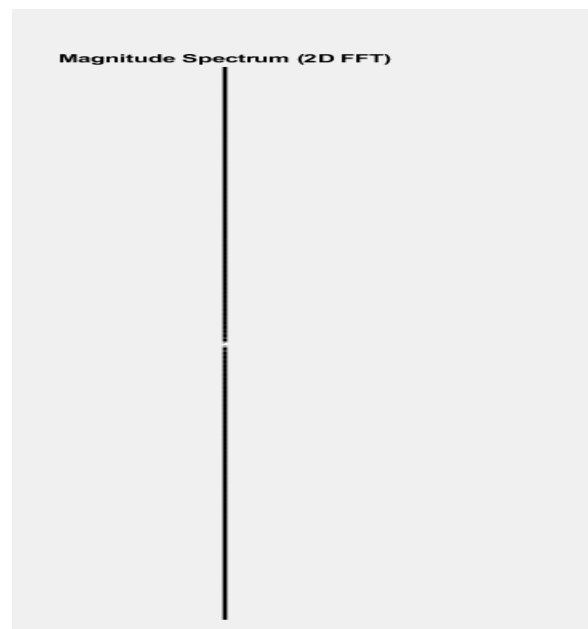
magnitude_normalized = uint8(magnitude / max(magnitude) * 255);

figure;
imshow(magnitude_normalized, []);
title('Magnitude Spectrum (2D FFT)');
```

شکل 2 برای سوال دوم



شکل 3 تصویر خروجی خواسته شده



شکل 4 تبدیل فوری تصویر قبل خروجی سوال 2

نوشتن تابع فیلتر :



```

39     n_order = 2*2;
40     if strcmp(filter_type, 'lowpass')
41         H = 1 ./ (1 + (D ./ D0).^(n_order));
42     else
43         H = 1 - 1 ./ (1 + (D ./ D0).^(n_order));
44     end
45     elseif strcmp(filter_name, 'gaussian')
46         if strcmp(filter_type, 'lowpass')
47             H = exp(-(D.^2) / (2*D0^2));
48         else
49             H = 1 - exp(-(D.^2) / (2*D0^2));
50         end
51     end
52
53     G = H .* F;
54
55     g = ifft2(ifftshift(G));
56     g = g(1:M_orig, 1:N_orig);
57
58     filtered_image = uint8(real(g));
59 end
60

```

```

1 function filtered_image = frequency_filter(image, filter_type, filter_name, D0)
2 f = double(image);
3 [M_orig, N_orig] = size(f);
4
5 M_pad = 2*M_orig - 1;
6 N_pad = 2*N_orig - 1;
7
8 pad_rows = M_pad - M_orig;
9 pad_cols = N_pad - N_orig;
10
11 pad_r1 = floor(pad_rows/2);
12 pad_r2 = pad_rows - pad_r1;
13 pad_c1 = floor(pad_cols/2);
14 pad_c2 = pad_cols - pad_c1;
15
16 f_padded = padarray(f, [pad_r1, pad_c1], 'post'); % Padding in one direction, the
17 f_padded = padarray(f_padded, [pad_r2, pad_c2], 'pre'); % Filling the rest
18
19 P = M_pad;
20 Q = N_pad;
21
22 f_padded = zeros(P, Q);
23 f_padded(1:M_orig, 1:N_orig) = f;
24
25 F = fftshift(fft2(f_padded));
26
27 [V, U] = meshgrid(0:Q-1, 0:P-1);
28 U = U ./ Q/2;
29 V = V ./ P/2;
30 D = sqrt(U.^2 + V.^2);
31 H = zeros(P, Q);
32 if strcmp(filter_name, 'ideal')
33     if strcmp(filter_type, 'lowpass')
34         H(D <= D0) = 1;
35     else
36         H(D > D0) = 1;
37     end
38 elseif strcmp(filter_name, 'butterworth')
39     n_order = 2*2;

```

توضیح:

## 1. آماده‌سازی و پدگذاری تصویر (Padding)

هدف: تبدیل تصویر به نوع داده double برای محاسبات دقیق‌تر و سپس پدگذاری (Padding) برای جلوگیری از Aliasing هنگام تبدیل فوریه و همچنین اطمینان از کارکرد صحیح محاسبه مرکز فرکانس (که مرکز طیف تبدیل فوریه است).

\* `f = double(image);` : تبدیل به double برای جلوگیری از سرریز (overflow) یا گرد کردن نادرست در محاسبات کسری.

\* `[M_orig, N_orig] = size(f);` : دریافت ابعاد اصلی تصویر.

\* محاسبات `M_pad` و `N_pad` : ابعاد پدگذاری شده (که اغلب به صورت مضربی از ابعاد اصلی یا به گونه‌ای انتخاب می‌شوند که مرکز طیف دقیقاً مشخص باشد).

\* `f_padded = zeros(P, Q); f_padded(1:M_orig, 1:N_orig) = f;` : این خطوط تصویر اصلی را در گوشه بالا-چپ ماتریس پد شده بزرگتر ( $P \times Q$ ) قرار می‌دهند. هدف این است که تصویر با صفرهای اضافی احاطه شود تا ابعاد FFT مشخص شود. برای تبدیل FFT صحیح در نرم‌افزار، معمولاً تصویر باید برای اینکه مرکز فرکانس (DC component) در مرکز طیف قرار گیرد، شیف‌ت داده شود (که بعداً توسط `fftshift` انجام می‌شود). قرار دادن تصویر در گوشه بالا-چپ، یک ناحیه فاز صفر (DC component) ایجاد می‌کند که با `fftshift` به مرکز منتقل می‌شود.

## 2. تبدیل فوریه (Fourier Transform)

\*  $F = \text{fftshift}(\text{fft2}(f\_padded))$  ;

\*  $\text{fft2}(f\_padded)$  : محاسبه تبدیل فوریه دو بعدی (DFT) روی تصویر پدگذاری شده. خروجی این تابع، طیف فرکانسی است که مولفه DC (فرکانس صفر) آن در گوشه بالا-چپ قرار دارد.

## 3. محاسبه طیف فرکانسی (D)

هدف: ایجاد یک ماتریس فاصله (D) که نشان‌دهنده فاصله هر نقطه فرکانسی از مرکز طیف (فرکانس صفر) است.

\*  $[V, U] = \text{meshgrid}(0:Q-1, 0:P-1)$  ; : ایجاد شبکه‌های مختصات برای مختصات ردیف (U) و ستون (V) در حوزه فرکانس با ابعاد  $P \times Q$ .

\*  $U = U - Q/2$ ;  $V = V - P/2$  ; : با توجه به اینکه مرکز طیف در  $\text{fftshift}$  به مرکز ماتریس منتقل شده است (تقریباً در  $Q/2$  و  $P/2$ )، این خطوط مختصات را نسبت به مرکز نرمال‌سازی می‌کنند. مختصات مرکزی صفر می‌شوند.

\*  $D = \text{sqrt}(U.^2 + V.^2)$  ; : محاسبه فاصله اقلیدسی (فاصله فرکانسی) هر نقطه از مرکز طیف با استفاده از قضیه فیثاغورث. این ماتریس (D) برای تعریف توابع فیلتر شعاعی (مانند ایده‌آل، باترورث، گوسی) ضروری است.

## 4. ساخت ماسک فیلتر (H)

هدف: تعریف تابع انتقال سیستم (Transfer Function) یا ماسک فیلتر (H) بر اساس نوع فیلتر درخواستی.

\*  $G = H .* F$  ; : ضرب در حوزه فرکانس. این عمل اساسی فیلترینگ خطی در حوزه فرکانس است. خاصیت کانولوشن در حوزه فوریه بیان می‌کند که کانولوشن در حوزه مکان معادل ضرب در حوزه فرکانس است.

\*  $g = \text{ifft2}(\text{ifftshift}(G))$  ; : تبدیل معکوس برای بازگرداندن تصویر به حوزه مکان.

\*  $\text{ifftshift}(G)$  : از آنجا که فیلتر (H) با توجه به طیف شیفت‌یافته (F) ساخته شده است، ابتدا باید طیف (G) را به حالت استاندارد (DC در گوشه بالا-چپ) بازگردانیم تا  $\text{ifft2}$  بتواند به درستی عمل کند.

\*  $\text{ifft2}(\dots)$  : محاسبه تبدیل فوریه دو بعدی معکوس.

\*  $g = g(1:M\_orig, 1:N\_orig)$  ; : برش تصویر بازسازی شده به ابعاد اصلی تصویر ورودی ( $M\_orig \times N\_orig$ )، زیرا تصویر پدگذاری شده بزرگتر است.

\*  $\text{filtered\_image} = \text{uint8}(\text{real}(g))$  ; : خروجی تبدیل معکوس ممکن است شامل اجزای موهومی بسیار کوچک ناشی از خطاهای محاسباتی باشد، بنابراین بخش حقیقی آن استخراج می‌شود و نهایتاً به نوع داده اصلی تصویر ( $\text{uint8}$ ) تبدیل می‌گردد.

## بارگزاری تصویر و گرفتن خروجی :

```
image_file = valid_a.tif ;
filter_type = 'lowpass';
D0_values = [10, 30, 150, 450];
filter_names = {'ideal', 'butterworth', 'gaussian'};

img_original = imread(image_file);
img_gray = img_original;
[M, N] = size(img_gray);

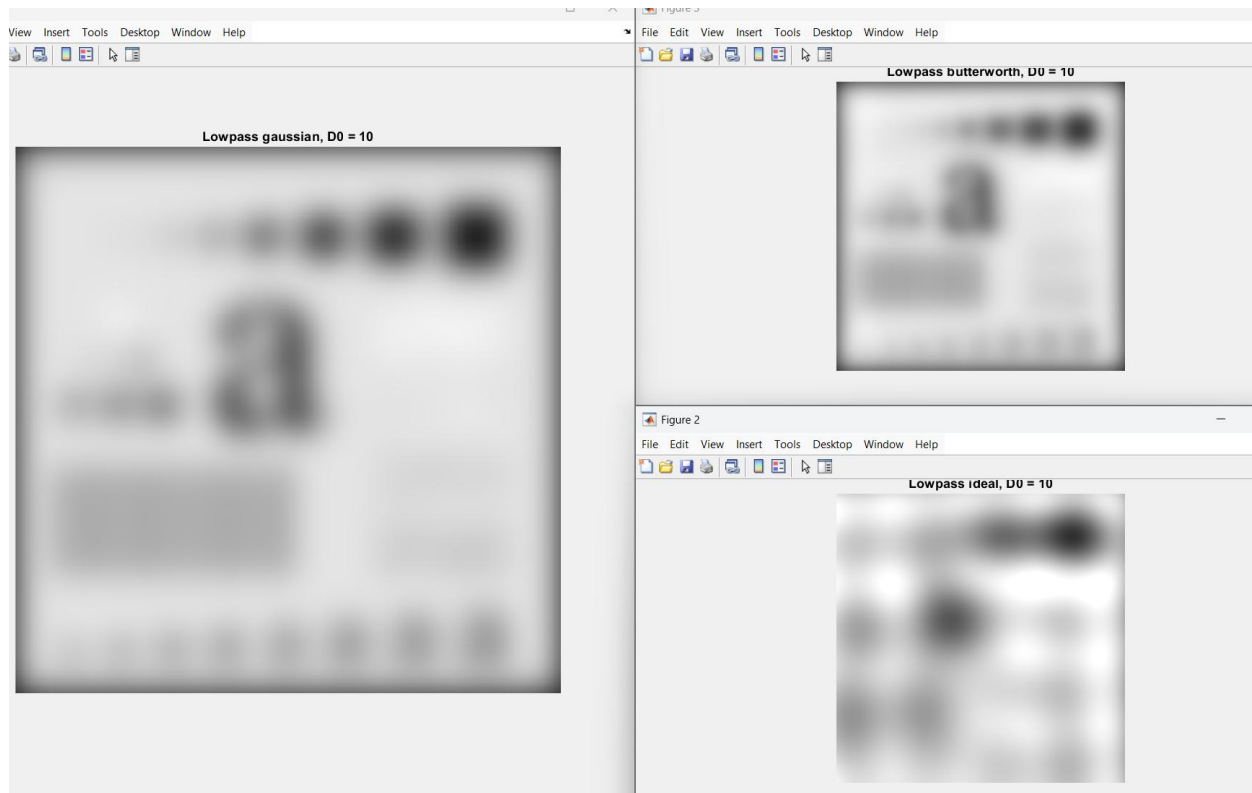
F_original = fftshift(fft2(padarray(img_gray, [M-1, N-1], 'both')));
figure;
imshow(log(1 + abs(F_original)), []);
title('طیف فوریة تصویر اصلی (Log Scale)');

for idx_d0 = 1:length(D0_values)
    D0 = D0_values(idx_d0);
    |
    for idx_filter = 1:length(filter_names)
        filter_name = filter_names{idx_filter};

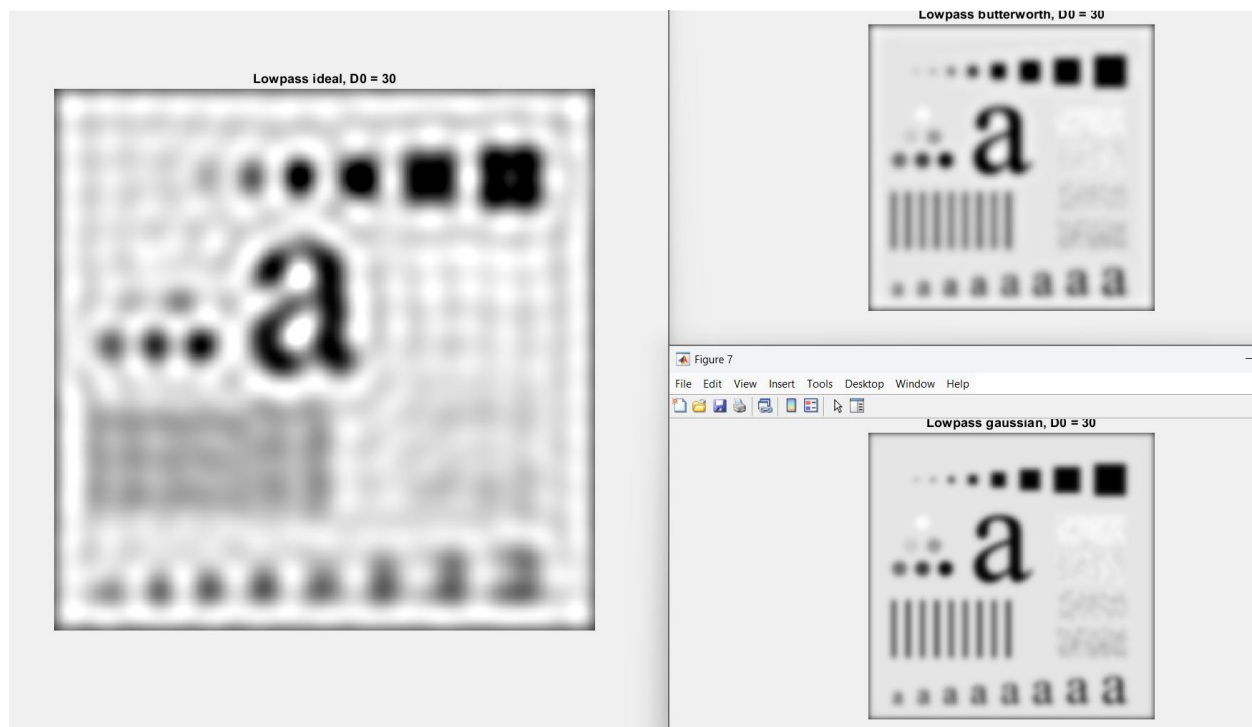
        filtered_img = frequency_filter(img_gray, filter_type, filter_name, D0);

        figure;
        imshow(filtered_img);
        title(sprintf('Lowpass %s, D0 = %d', filter_name, D0));
    end
end
```

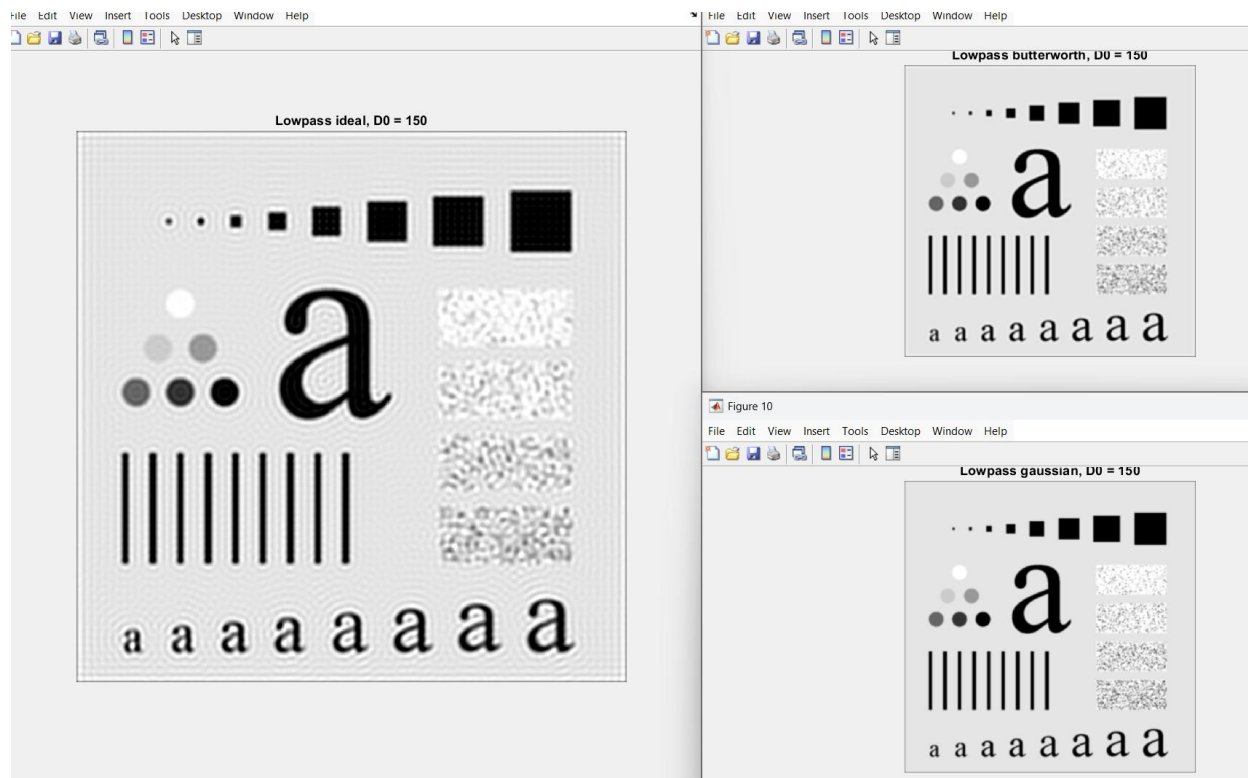
خروجی برای  $D0 = 10$  :



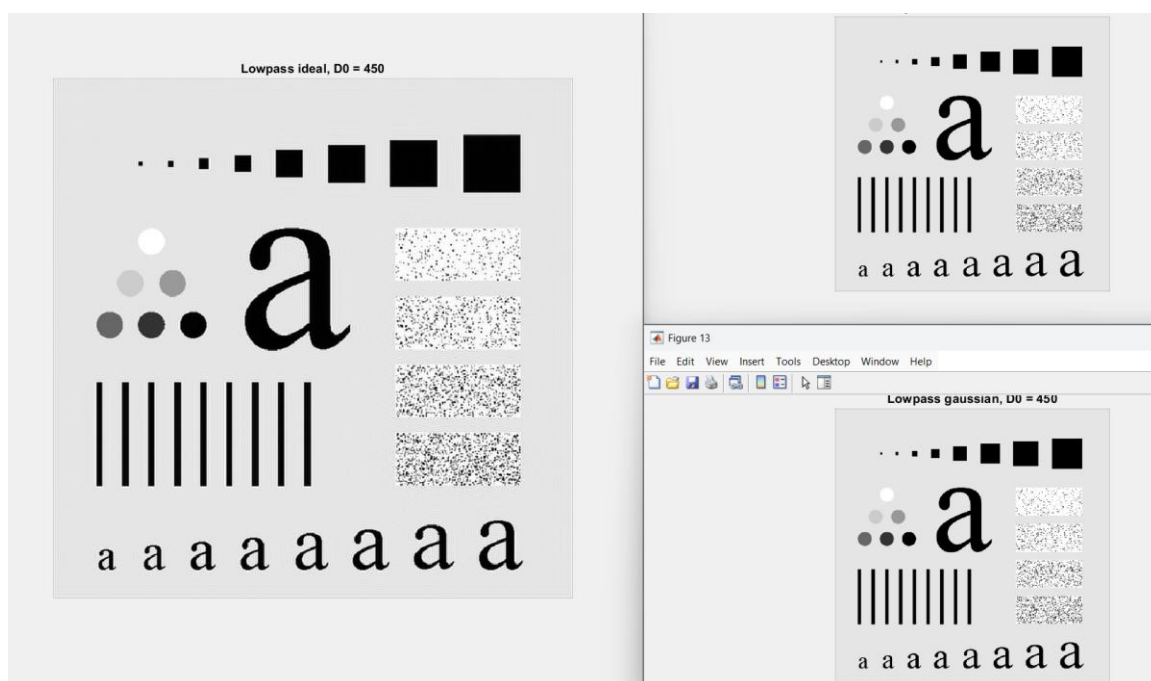
خروجی برای  $D0 = 30$  :



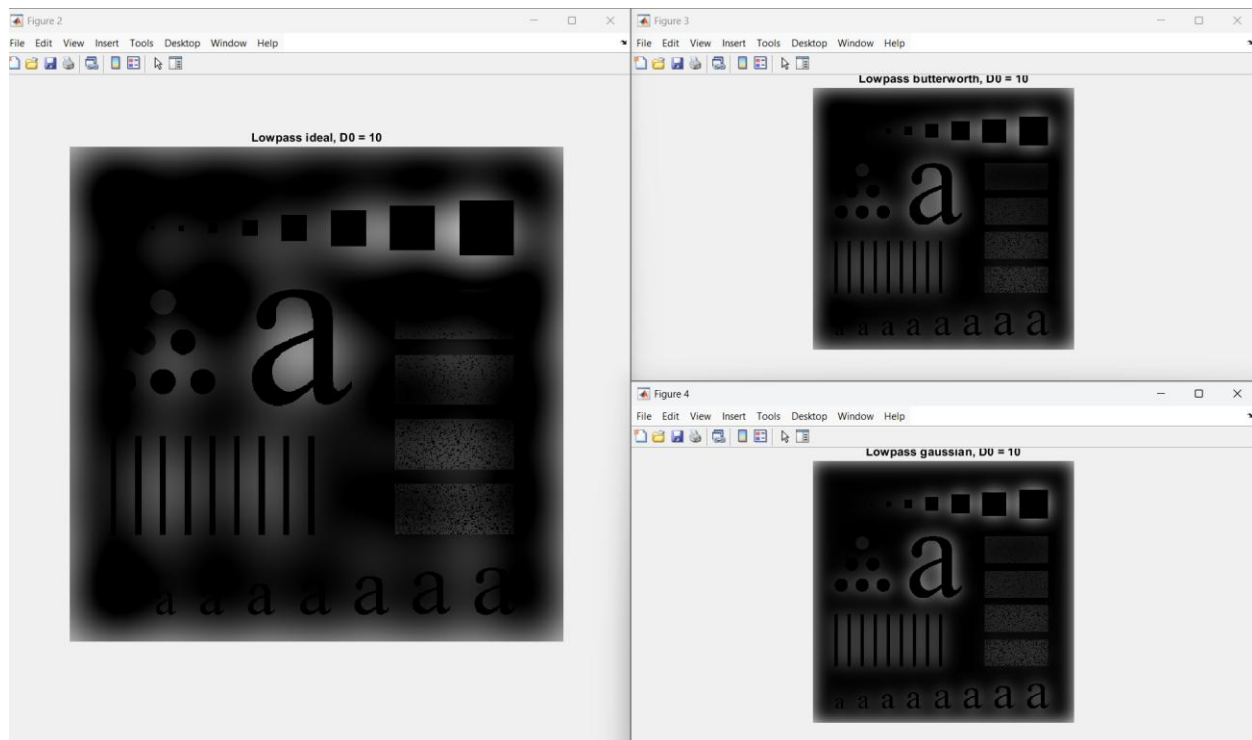
خروجی برای  $D0 = 150$  :



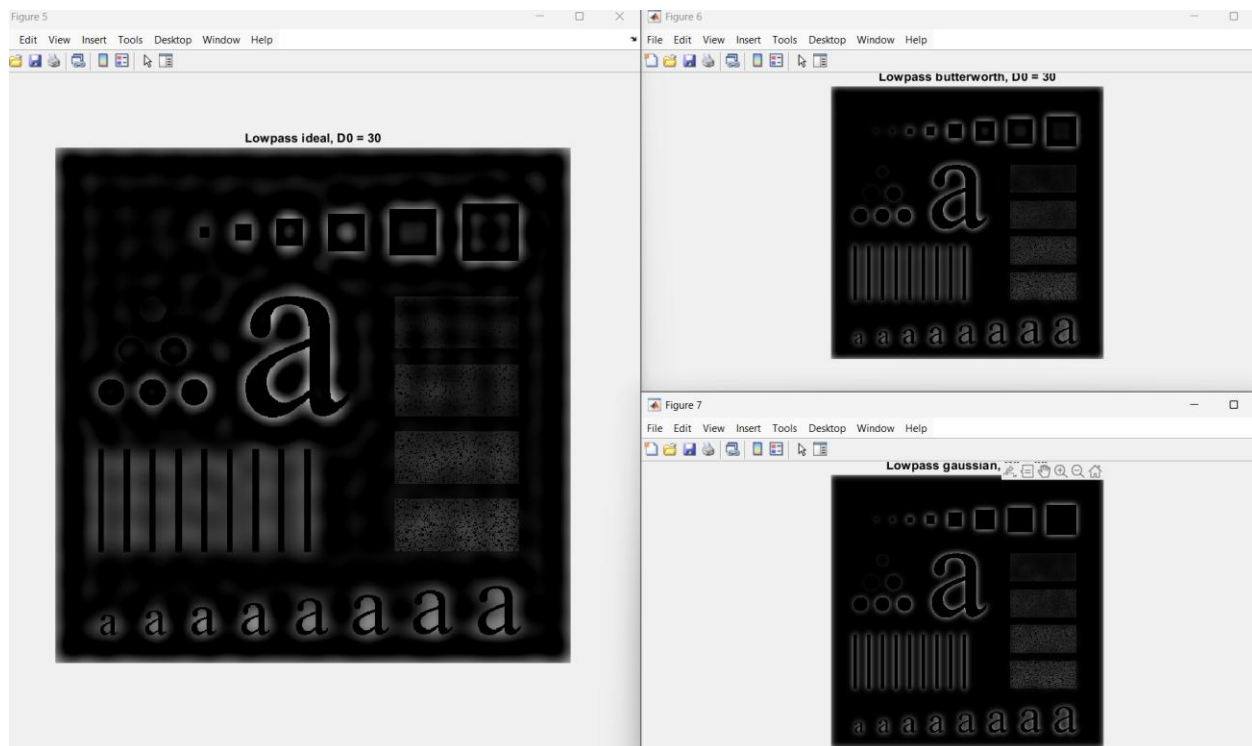
خروجی برای  $D_0 = 450$  :



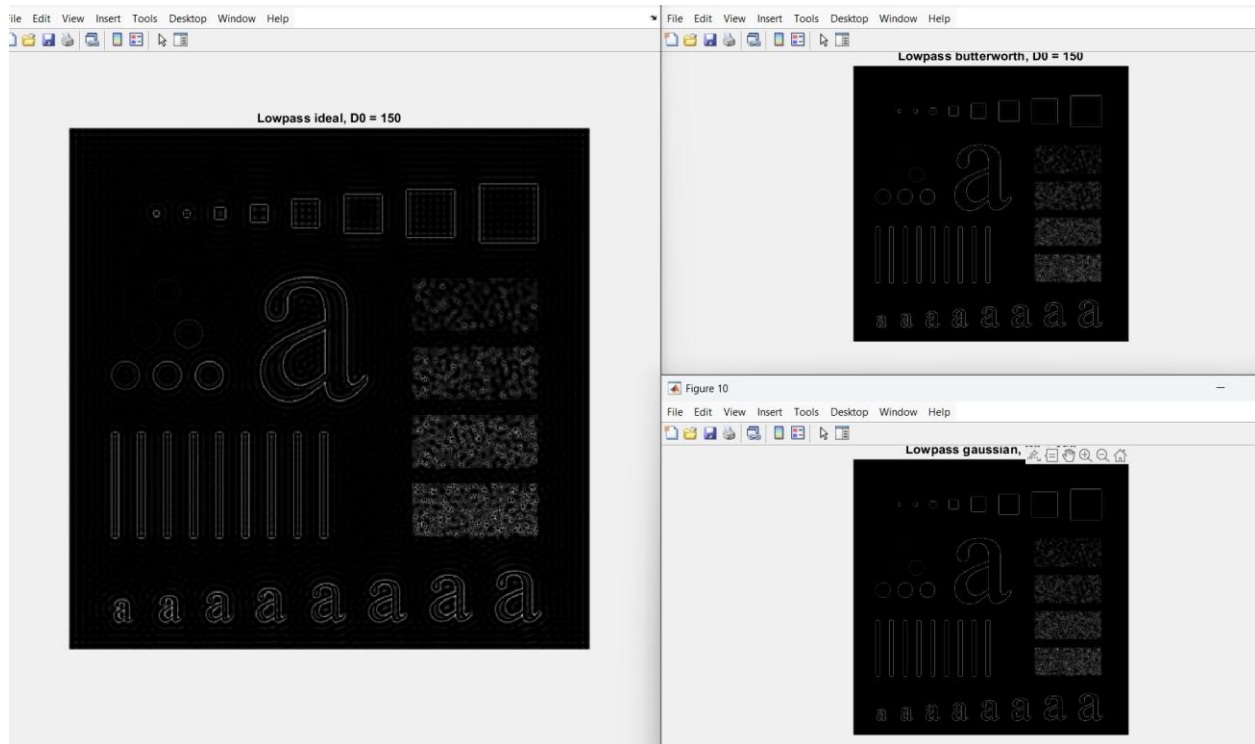
و خروجی بالاگذر برای  $D_0 = 10$  :



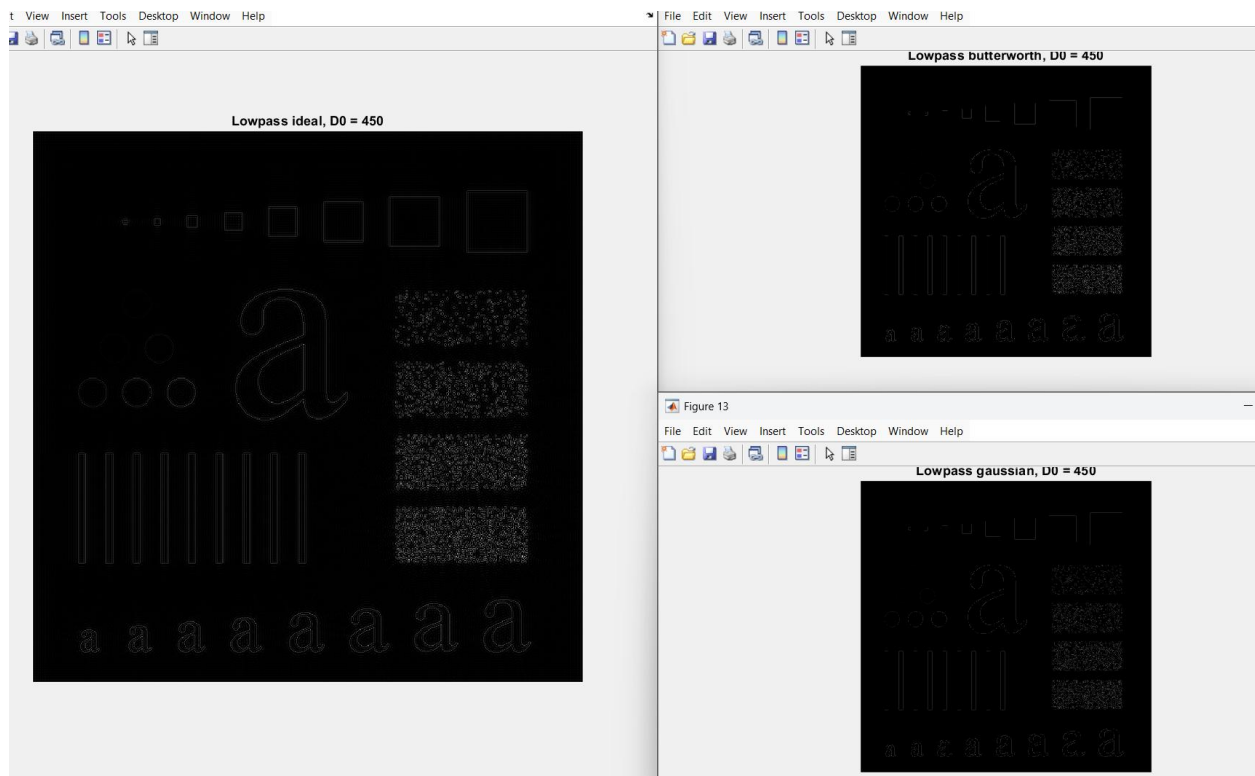
خروجی بالاگذر برای  $D_0 = 30$  :



خروجی بالاگذر برای  $D0 = 150$



خروجی بالاگذر برای  $D0 = 450$  :



مقایسه و توضیح:

- در تصویر، تغییرات آرام نماینده فرکانس‌های پایین هستند.
- تغییرات سریع (مانند لبه‌ها یا نویز) نماینده فرکانس‌های بالا هستند.
- فیلترهای پایین‌گذر (LPF) فرکانس‌های بالا را تضعیف می‌کنند و تصویر را صاف می‌کنند.
- فیلترهای بالاگذر (HPF) فرکانس‌های پایین را تضعیف می‌کنند و لبه‌ها و جزئیات را برجسته می‌کنند.

2. انواع فیلترها:

- Ideal: انتقال ناگهانی، منجر به Ringing می‌شود.
- Butterworth: انتقال نرم‌تر، کاهش Ringing.
- Gaussian: انتقال نرم و یکنواخت، کمترین اثر Ringing.

3. تحلیل اثر فیلترها:

الف) فیلترهای پایین‌گذر (LPF):

- $D0 = 10$ : تصویر بسیار تار می‌شود، لبه‌ها و جزئیات اصلی حذف می‌شوند.
- $D0 = 30$ : تاری متوسط، جزئیات کمی حفظ می‌شوند.
- $D0 = 150$ : تاری کمتر، لبه‌ها و جزئیات بیشتری حفظ می‌شوند.
- $D0 = 450$ : کمترین تاری، تصویر نزدیک به اصلی است.

ب) فیلترهای بالاگذر (HPF):

- $D0 = 10$ : لبه‌ها بسیار تیز و برجسته، نویز زیاد.
- $D0 = 30$ : لبه‌ها برجسته، نویز کم.
- $D0 = 150$ : لبه‌ها برجسته، نویز نسبتاً کم.
- $D0 = 450$ : کمترین لبه‌گذاری، تصویر نزدیک به اصلی است.

4. مقایسه فیلترها:

:LPF

- Ideal: تاری شدید، اما Ringing قابل توجه.
- Butterworth: تاری خوب، کمتر Ringing.
- Gaussian: تاری نرم، بهترین کیفیت.



- HPF :

- Ideal : برجسته‌سازی تیز، نویز زیاد.

- Butterworth : برجسته‌سازی متوسط، نویز کم.

- Gaussian : برجسته‌سازی نرم، کمترین نویز.

چرا پس از FFT از fftshift استفاده میکنیم :

1. استاندارد نمایش طیف فرکانس: تابع  $\text{fft2}$  در متلب (و بسیاری از پیاده‌سازی‌های DFT) مولفه فرکانس صفر (DC component) ، یعنی میانگین تصویر، را در گوشه بالا-چپ ماتریس خروجی قرار می‌دهد.

2. نیاز به مرکزیت فرکانس: برای اعمال فیلترهای شعاعی (مانند فیلترهای ایده‌آل، گاوسی یا باترورت) بر اساس فرکانس قطع ، باید مرکز طیف (فرکانس صفر) در مرکز هندسی ماتریس قرار گیرد.

$\text{fftshift}$  طیف را جابجا می‌کند تا نمایش آن از حالت گوشه بالا-چپ به نمایش مرکزیت تغییر یابد، که این حالت برای اعمال فیلترهای متقارن شعاعی و تفسیر بصری طیف ضروری است.

چرایی استفاده از Zero Padding در فیلترینگ حوزه فرکانس

هنگامی که شما یک تصویر  $f(x, y)$  با ابعاد  $N \times M$  دارید و می‌خواهید آن را در حوزه فرکانس فیلتر کنید. مشکل اصلی زمانی پیش می‌آید که ابعاد تصویر اصلی  $(N \times M)$  و ابعاد فیلتر ایده‌آل  $(P \times Q)$  با هم ناهماهنگ باشند.

### 1. اصل همبستگی (Convolution Theorem)

قضیه همبستگی بیان می‌کند که تبدیل فوریه کانولوشن (پیچش) دو تابع در حوزه مکان، برابر است با ضرب توابع مربوطه در حوزه فرکانس:

$$\text{DFT}\{f(x,y)*h(x,y)\}=F(u,v)\cdot H(u,v)$$

برای اینکه عملیات ضرب در حوزه فرکانس معادل عملیات کانولوشن در حوزه مکان باشد ابعاد تصویر و فیلتر باید یکسان باشد یا تصویر باید با صفرها پد (Padding) شود تا ابعاد آن به اندازه کافی بزرگ شود.

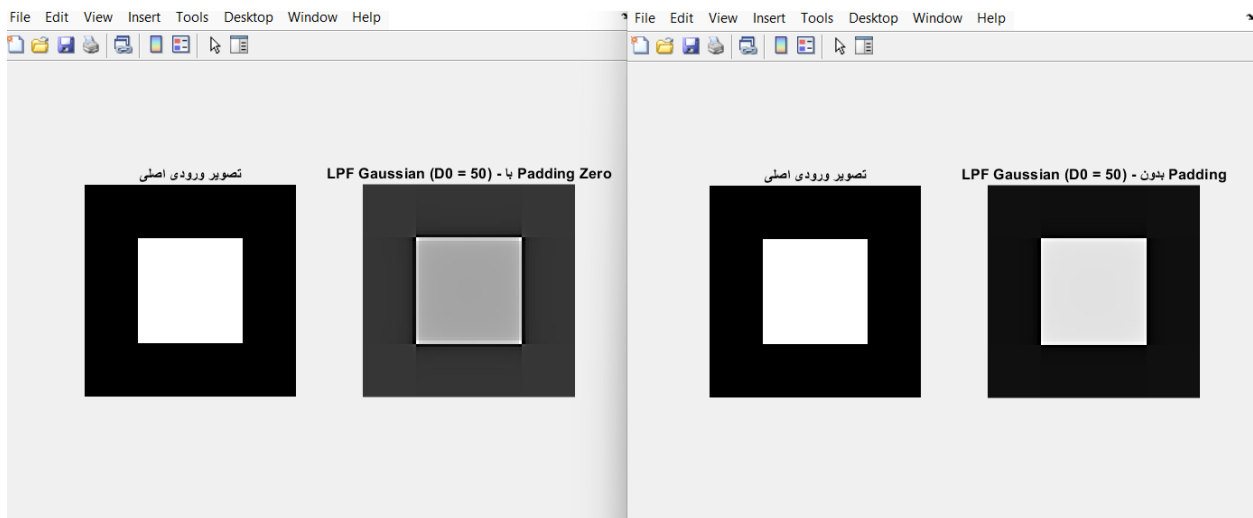
### 2. اثر عدم استفاده از Zero Padding (وقتی ابعاد نامساوی است)

اگر ابعاد تصویر اصلی با ابعاد فیلتر ایده‌آل برابر نباشد:

\* نتیجه نادرست کانولوشن (Aliasing): وقتی IDFT را روی حاصل ضرب  $F(u,v)\cdot H(u,v)$  اعمال می‌کنید، اگر ابعاد فیلتر بزرگتر از تصویر باشد، یا اگر اندازه فیلتر به طور مناسبی انتخاب نشده باشد، شما عملاً عملیات کانولوشن دوره‌ای را انجام می‌دهید. این کار باعث می‌شود اطلاعات لبه‌های تصویر که از یک طرف خارج می‌شوند، از طرف دیگر وارد شوند (پدیده Wraparound یا Aliasing).

\* لبه‌های تیز تصویر: در فیلترهایی مانند فیلتر بالاگذر (High-pass) که لبه‌ها را تقویت می‌کنند، عدم پدینگ باعث می‌شود اثر فیلتر به صورت یک نویز تکرار شونده در مرزهای تصویر ظاهر شود.

به عبارت ساده‌تر، تصویر باید با صفرها پد شود تا ابعاد آن حداقل برابر با مجموع ابعاد تصویر و فیلتر منهای یک باشد.



کد :

```
function filtered_img = frequency_filter_process(img, D0, use_padding)
[m, n] = size(img);

if use_padding
    P = 2*m - 1;
    Q = 2*n - 1;
    padded_img = zeros(P, Q);
    padded_img(1:m, 1:n) = img;

    F = fft2(padded_img);
    H = create_gaussian_filter(P, Q, D0);
else
    F = fft2(img);
    H = create_gaussian_filter(m, n, D0);
end

G = F .* H;
g = ifft2(G);

if use_padding
    filtered_img = real(g(1:m, 1:n));
else
    filtered_img = real(g);
end

min_val = min(filtered_img(:));
max_val = max(filtered_img(:));

if max_val - min_val > 1e-9
    filtered_img = (filtered_img - min_val) / (max_val - min_val);
end
end
```

```
filtered_img_no_pad = frequency_filter_process(img_gray, D0, false);

figure;
subplot(1, 2, 1);
imshow(img_gray);
title('تصویر ورودی اصلی');

subplot(1, 2, 2);
imshow(filtered_img_no_pad);
title(sprintf('LPF Gaussian (D0 = %d) - بدون Padding', D0));

filtered_img_with_pad = frequency_filter_process(img_gray, D0, true);

figure;
subplot(1, 2, 1);
imshow(img_gray);
title('تصویر ورودی اصلی');

subplot(1, 2, 2);
imshow(filtered_img_with_pad);
title(sprintf('LPF Gaussian (D0 = %d) - با Padding Zero', D0));

|
```